



Chapter 9

Stacks and Queues

9.1 Introduction to Stacks

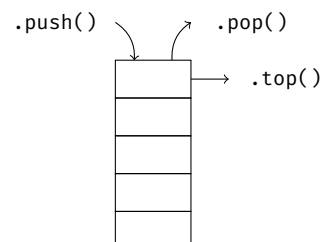
A **stack** is a one-ended linear data structure that supports last-in, first-out (LIFO) data access. When you retrieve elements out of a stack, you remove the elements in the opposite order in which they were inserted.

For a real-life example of how a stack works, consider a stack of books on a table. When you add a book to the stack, you add it to the very top of the pile. When you retrieve a book from the stack, you must remove the book on top — the one you most recently added. You cannot access the book at the very bottom of the stack without removing all the other books on top of it.

A stack works in a similar manner. When you call `.push()`, you add an element to the top of the stack. When you call `.top()`, you retrieve the most recently added element out of the stack. When you call `.pop()`, you remove the most recently added element from the stack.

The interface of a stack:

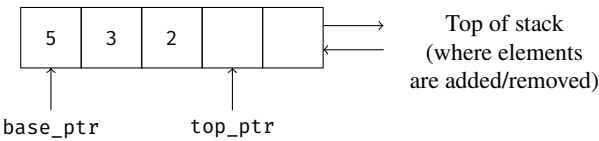
Function	Behavior
<code>.push(val)</code>	Adds <code>val</code> to the top of the stack
<code>.pop()</code>	Removes top element from the stack
<code>.top()</code>	Returns a reference to the top element in the stack
<code>.size()</code>	Returns the number of elements in the stack
<code>.empty()</code>	Returns whether the stack is empty



Stacks have numerous practical use cases in programming. For instance, text editors use a stack's behavior to implement "undo" buttons that allow you to revert changes. Compilers also use stacks to check for matching parentheses in code (e.g., making sure every `'{'` has a matching `'}'` in the right place). Stacks also play an important role in recursion (by keeping track of the data of previous function calls) and graph-searching algorithms (such as a depth-first search, which will be covered in chapter 19).

※ 9.1.1 Implementing a Stack Using an Array

How would you implement a stack container in memory? One strategy would be to use an array as the underlying container and keep a pointer to the first empty position past the last element:



If you want to push an element into the stack, you would add the element to the position pointed to by `top_ptr` and then increment the `top_ptr` pointer (allocating new space if needed). If no reallocation is needed, the time complexity of a single `.push()` is $\Theta(1)$. If reallocation is involved, all n elements have to be copied over, so the time complexity would be $\Theta(n)$, where n is the size of the stack.

If you want to pop an element off the stack, you would just need to decrement `top_ptr`. This takes $\Theta(1)$ time.

If you want to retrieve the top element, you would just need to dereference `top_ptr - 1`. This takes $\Theta(1)$ time. Note that we could check whether an element actually exists in the stack beforehand, but this check is omitted by the STL's `std::stack<>` implementation. This is for the sake of efficiency; the `.top()` operation would be less performant if a validity check were made every time it is called. By omitting the validity check, we leave it to the programmer to ensure that they are not calling `.top()` on an empty stack.

If you want to compute the size of the stack, you could return `top_ptr - base_ptr`. Because elements in an array are contiguous in memory, the difference between the two pointers is equal to the number of elements in the array. This operation takes $\Theta(1)$ time.

If you want to check whether the stack is empty, you could check if `base_ptr` and `top_ptr` are equal. Because `top_ptr` points to the first available position in the stack, if `top_ptr` points to the first position, the first position must be empty. This check also takes $\Theta(1)$ time.

In summary, if you implement a stack using an array as the underlying container, you can achieve the following time complexities:

Method	Implementation	Complexity
<code>.push(val)</code>	Add new element at the position pointed to by <code>top_ptr</code> , then increment <code>top_ptr</code>	$\Theta(1)$ if no reallocation, $\Theta(n)$ if reallocation involved ¹
<code>.pop()</code>	Decrement <code>top_ptr</code>	$\Theta(1)$
<code>.top()</code>	Dereference <code>top_ptr - 1</code>	$\Theta(1)$
<code>.size()</code>	Calculate <code>top_ptr - base_ptr</code>	$\Theta(1)$
<code>.empty()</code>	Check if <code>base_ptr == top_ptr</code>	$\Theta(1)$

The code for the array-based stack implementation is shown below:

```
1  template <typename T>
2  class Stack {
3      T* data;           // pointer to array
4      T* top_ptr;        // pointer to first open position
5      size_t capacity;   // array capacity
6  public:
7      Stack(size_t num = 4); // array capacity defaults to 4 if not specified
8      ~Stack();
9      void push(T val);
10     void pop();
11     T& top();
12     size_t size();
13     bool empty();
14 };
15
16 template <typename T>
17 Stack<T>::Stack(size_t num) : capacity{num} {
18     data = (num ? new T[num] : nullptr); // allocate array memory
19     top_ptr = data;                       // set top_ptr to top of data
20 } // Stack()
21
22 template <typename T>
23 Stack<T>::~~Stack() {
24     delete[] data;
25 } // ~Stack()
```

¹Using amortized analysis, `.push()` is an amortized $\Theta(1)$ operation. This concept will be discussed in chapter 12.

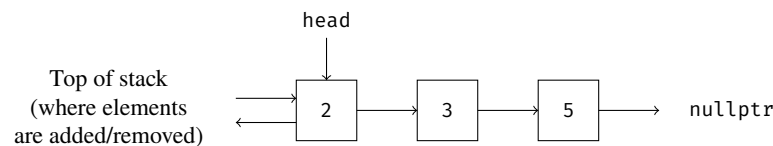
```

27  template <typename T>
28  void Stack<T>::push(T val) {
29      if (top_ptr - data >= capacity) {          // if current array is full
30          T* temp = new T[capacity * 2];        // double capacity of array
31          for (size_t i = 0; i < capacity; ++i) { // copy elements over
32              temp[i] = data[i];
33          } // for i
34          top_ptr = &temp[capacity];            // reset top_ptr to new array
35          capacity *= 2;                        // update capacity value
36          std::swap(temp, data);                // set data to new array
37          delete[] temp;                        // deallocate old array
38      } // if
39      *top_ptr++ = val;                          // writes new value to top_ptr
40  } // push()
41
42  template <typename T>
43  void Stack<T>::pop() {
44      --top_ptr;
45  } // pop()
46
47  template <typename T>
48  T& Stack<T>::top() {
49      return *(top_ptr - 1);
50  } // top()
51
52  template <typename T>
53  size_t Stack<T>::size() {
54      return top_ptr - data;
55  } // size()
56
57  template <typename T>
58  bool Stack<T>::empty() {
59      return data == top_ptr;
60  } // empty()

```

※ 9.1.2 Implementing a Stack Using a Linked List

We could also use a linked list to implement our stack. A singly-linked list is sufficient for implementing a stack — there is no need to store prev pointers, since we only need to operate on values on one end of the stack. In our linked list, we will treat the head as the top of the stack. This is because it is more efficient to add and remove elements from the head than it is to add and remove elements from the tail.



In this case, if we want to push an element into the stack, we would insert it at the beginning of the list. The amount of work required to insert an element at the beginning of a list does not depend on the size of the list, so the time complexity of pushing an element is $\Theta(1)$.

If we want to pop an element off the stack, we would delete the element at the head pointer. This requires us to set the head pointer to head->next and deallocate the old head. Since we are only moving some pointers around, this takes $\Theta(1)$ time.

If we want to retrieve the top element, we would have to return a reference to the data stored in the head node. Since we already have access to the head node, we can access its data in $\Theta(1)$ time.

If we want to return the size of the stack, we could count the number of nodes each time size is called; this would result in a $\Theta(n)$ operation. However, we could also store an additional member variable that stores the size. If we keep track of the size internally, we would not need to traverse the list each time we want the size of the stack; we could just check the value of our size variable. Even though this approach adds a bit more work to other operations (insertions and deletions would have to modify size), it allows us to retrieve the size of the list in $\Theta(1)$ time.

If we want to check if the stack is empty, we could just check to see if head == nullptr (or check if size == 0 variable if we have a separate size variable). This check takes $\Theta(1)$ time.

In summary, if you implement a stack using a list as the underlying container, you can achieve the following time complexities:

Method	Implementation	Complexity
.push(val)	Insert val to the front of list	$\Theta(1)$
.pop()	Delete node at the front of list	$\Theta(1)$
.top()	Return reference to data of head node	$\Theta(1)$
.size()	Track size internally as a member variable, or count nodes each time	$\Theta(1)$ if size stored internally, $\Theta(n)$ if nodes counted every time
.empty()	Check if head is nullptr	$\Theta(1)$

The code for the list-based stack implementation is shown below:

```
1  template <typename T>
2  class Stack {
3      struct Node {
4          T val;
5          Node* next;
6          Node(T val_in) : val{val_in}, next{nullptr} {}
7      };
8      Node* head;
9      size_t sz;
10 public:
11     Stack();
12     ~Stack();
13     void push(T val);
14     void pop();
15     T& top();
16     size_t size();
17     bool empty();
18 };
19
20 template <typename T>
21 Stack<T>::Stack() : head{nullptr}, sz{0} {}
22
23 template <typename T>
24 Stack<T>::~~Stack() {
25     Node* temp;
26     while (head != nullptr) {                // destructor goes through
27         temp = head->next;                    // list and deallocates
28         delete head;                          // all the nodes
29         head = temp;
30     } // while
31 } // ~Stack()
32
33 template <typename T>
34 void Stack<T>::push(T val) {
35     Node* new_node = new Node{val};          // insert node at head
36     new_node->next = head;
37     head = new_node;
38     ++sz;
39 } // push()
40
41 template <typename T>
42 void Stack<T>::pop() {                        // remove node at head
43     Node* temp = head;
44     head = temp->next;
45     --sz;
46     delete temp;
47 } // pop()
48
49 template <typename T>
50 T& Stack<T>::top() {
51     return head->val;
52 } // top()
53
54 template <typename T>
55 size_t Stack<T>::size() {
56     return sz;
57 } // size()
58
59 template <typename T>
60 bool Stack<T>::empty() {
61     return head == nullptr;
62 } // empty()
```

Which stack implementation is better, the one that uses an array or the one that uses a linked list? It turns out that both containers support all stack operations in $\Theta(1)$ time. However, if you were to time these implementations, you would likely find that the array implementation is slightly more performant than the linked list implementation. This is because, even though both implementations run in $\Theta(1)$ time, the constant factor of the array implementation is smaller. The linked list implementation also has higher memory overhead compared to the array (since additional pointers need to be stored with each element).

9.2 The STL Stack Container

The C++ standard template library provides a pre-implemented stack container for you in the `<stack>` library. To use this stack in your program, you will need to `#include <stack>` at the top of your code file and declare an object of type `std::stack<>`. A `std::stack<>` supports the following operations:

Function	Behavior
<code>.push(val)</code>	Adds <code>val</code> to the top of the stack
<code>.pop()</code>	Removes top element from the stack (undefined behavior if empty)
<code>.top()</code>	Returns a reference to the top element in the stack (undefined behavior if empty)
<code>.size()</code>	Returns the number of elements in the stack
<code>.empty()</code>	Checks if the stack is empty

In C++, the `.top()` operation returns a reference to the top element in the stack but does not remove it. The `.pop()` operation removes the top element but does not return it. If you want to retrieve *and* remove the top element in a stack, you must call both `.top()` and `.pop()`.

The following code provides an example that utilizes a `std::stack<>`:

```
1  std::stack<int32_t> s;           // initializes a stack with variable name 's'
2  s.push(5);                      // pushes 5 onto the stack
3  s.push(3);                      // pushes 3 onto the stack
4  int32_t x = s.top();            // x stores the value 3
5  s.pop();                        // 3 is removed from the stack
6  s.top() = 4;                   // the top element is changed from 5 to 4
7  s.pop();                        // 4 is removed from the stack
8  std::cout << s.size() << '\n'; // stack is empty, so this prints out 0
```

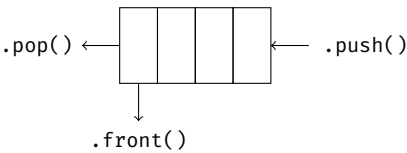
9.3 Introduction to Queues

A **queue** is linear data structure that supports first-in, first-out (FIFO) data access. When you retrieve elements out of a queue, you remove the elements in the same order in which you inserted them.

For a real-life example of how a queue works, think of an office hours queue where students wait in line for help. When a student adds themselves to the queue, they put themselves at the end of the line. If an instructor is available, they get the student who has been in the queue the longest. A queue works the same on data. When you *enqueue* an element, you add the element to the *back* of the queue. When you *dequeue* an element, you remove the element at the *front* of the queue. In C++, elements are enqueued using `.push()` and dequeued using `.pop()`.

The interface of a queue:

Function	Behavior
<code>.push(val)</code>	Adds <code>val</code> to the back of the queue
<code>.pop()</code>	Removes top element from the front of the queue
<code>.front()</code>	Returns a reference to the element at the front of the queue
<code>.size()</code>	Returns the number of elements in the queue
<code>.empty()</code>	Checks if the queue is empty

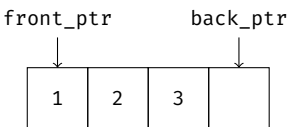


It is important to remember that stacks in the STL use `.top()` to retrieve the next element, while queues in the STL use `.front()` to retrieve the next element. The other method names of the two containers are identical.

※ 9.3.1 Implementing a Queue Using an Array (Circular Buffer)

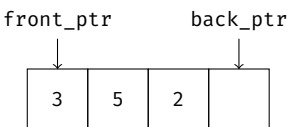
Like with stack, a queue can be implemented using an array or a linked list as the underlying structure. However, the implementation of a queue is slightly more complicated, since operations need to be supported on *both* ends of the underlying container. One method for implementing a queue using an array is to use a **circular buffer**. In a circular buffer, we allow elements to loop off the end of the array to support efficient deletions and insertions on opposing ends of the queue. To track the order of elements in a circular buffer, two pointers are used: `front_ptr`, which points to the element at the front of the queue, and `back_ptr`, which points to the first open position at the back of the queue.

Consider the following array, which represents our circular buffer:

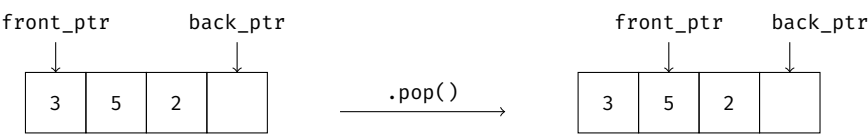


Here, the value 1 is at the front of the queue (and is next to be popped off), and the value 3 is at the back of the queue (the element most recently added to the queue). We know this because of the positions of `front_ptr` and `back_ptr`, which indicate the two ends of our queue.

Because a queue allows elements to be modified at both ends of the container, we have to be a bit more careful when storing our elements. With an array-based stack implementation, we knew that if `top_ptr` reached the end of the array, the stack was full and we needed to allocate more space. This assumption cannot be made with a queue, however, since the first element is not guaranteed to be in the leftmost position of the array! For instance, if we popped an element off the above queue, 1 would get removed and the first element would be located at the *second* position of the array. This is where the circular nature of the circular buffer comes in. Every time an element is pushed in, we add the element to the position pointed to by `back_ptr` and increment `back_ptr` by one. However, if `back_ptr` ends up off the end of the array, we *wrap around* back to the beginning of the array and check if the position is available. This process is illustrated below:

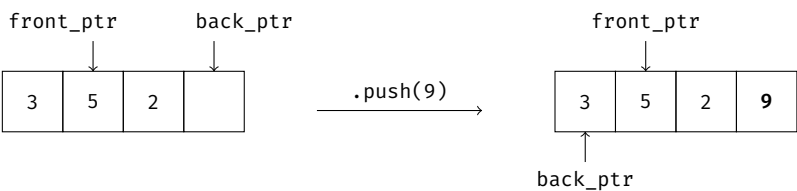


Suppose we call `.pop()`, which removes the element at the front of the queue. Since `front_ptr` is a pointer that points to the element at the front of the queue, we can "remove" the front element by moving `front_ptr` forward one position.

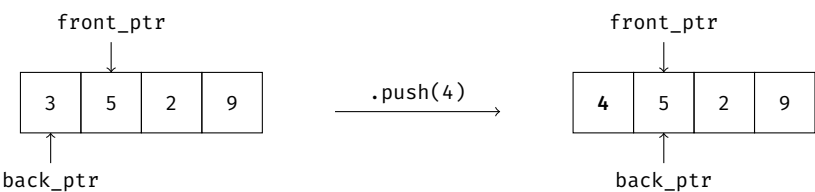


Notice here that the value 3 was not physically deleted from the array. This is because our queue, if implemented correctly, has no way of accessing this 3 since it is no longer a valid entry in our queue (which only looks at the values between the front and back pointers). As a result, there is no need to do the additional work of clearing this value out, as the queue should never access it anyway. We will only reset this value when we need to access index 0 of our array again.

Now, suppose we push the value 9 to the back of the queue by making a call to `.push(9)`. Since `back_ptr` points to the first open position in the queue, we can write 9 to this location and increment `back_ptr` by one position. Since a circular buffer wraps around, `back_ptr` ends up pointing to index 0 of the array.



If were to push the value 4 into the queue (by calling `.push(4)`), we would write 4 to the value pointed to by `back_ptr` and increment `back_ptr` forward one position. This is shown below.



With this procedure, `front_ptr` will always point to the element at the front of the queue, and `back_ptr` will always point to the next available position in the array. If incrementing `back_ptr` causes it to point to the same position as `front_ptr`, that means the array is filled to capacity, and reallocation would be needed to support any more elements. During reallocation, the elements in the array are copied to a larger array from front to back, so that the front element ends up at the first position of the new array. For example, if reallocation were done on the above array, the elements would be copied to a new array in the order [5, 2, 9, 4], since 5 is the element at the front.

The operations for a circular buffer, along with their complexities, are shown below:

Method	Implementation	Complexity
<code>.push(val)</code>	Insert <code>val</code> at position pointed to by <code>back_ptr</code> and increment <code>back_ptr</code> , wrapping around to the beginning of the array if necessary. If <code>back_ptr</code> is ever incremented to the position of <code>front_ptr</code> , a larger array is allocated and elements are copied over in order from front to back.	$\Theta(1)$ if no reallocation, $\Theta(n)$ if reallocation involved ²
<code>.pop()</code>	Increment <code>front_ptr</code>	$\Theta(1)$
<code>.front()</code>	Dereference <code>front_ptr</code> and return its value	$\Theta(1)$
<code>.size()</code>	If the address of <code>back_ptr</code> is larger than that of <code>front_ptr</code> , return the difference between the two pointers; otherwise, add the capacity of the array to the difference (i.e., return <code>array capacity + back_ptr - front_ptr</code>)	$\Theta(1)$
<code>.empty()</code>	Check if <code>back_ptr == front_ptr</code>	$\Theta(1)$

²Using amortized analysis, `.push()` is an amortized $\Theta(1)$ operation. This concept will be discussed in chapter 12.

The following code implements a queue using a circular buffer:

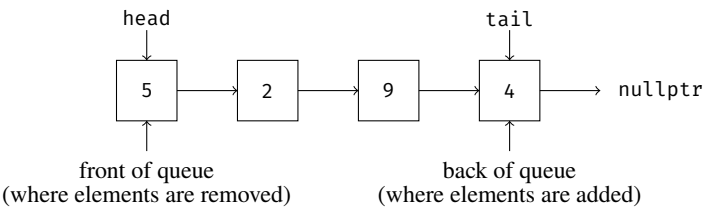
```

1  template <typename T>
2  class Queue {
3      T* data;
4      T* front_ptr;
5      T* back_ptr;
6      size_t capacity;
7  public:
8      // array capacity defaults to 4 if not specified
9      Queue(size_t num = 4);
10     ~Queue();
11     void push(T val);
12     void pop();
13     T& front();
14     size_t size();
15     bool empty();
16 };
17
18 template <typename T>
19 Queue<T>::Queue(size_t num) : capacity{num} {
20     data = (num ? new T[num] : nullptr);
21     front_ptr = back_ptr = data;
22 } // Queue()
23
24 template <typename T>
25 Queue<T>::~~Queue() {
26     delete[] data;
27 } // ~Queue()
28
29 template <typename T>
30 void Queue<T>::push(T val) {
31     *back_ptr++ = val; // write val to back_ptr
32     if (back_ptr >= data + capacity) { // loop around if off end
33         back_ptr = data;
34     } // if
35     if (back_ptr == front_ptr) { // if back hits front, reallocate
36         T* temp = new T[capacity * 2]; // double capacity of array
37         for (size_t i = 0; i < capacity; ++i) {
38             temp[i] = *front_ptr++; // write data starting from front
39             if (front_ptr == data + capacity) { // loop around if off end
40                 front_ptr = data;
41             } // if
42         } // for i
43         front_ptr = temp; // reset front_ptr to new array
44         back_ptr = &temp[capacity]; // reset back_ptr to new array
45         capacity *= 2; // update capacity
46         std::swap(temp, data); // set data to new array
47         delete[] temp; // deallocate old array
48     } // if
49 } // push()
50
51 template <typename T>
52 void Queue<T>::pop() {
53     ++front_ptr;
54     if (front_ptr >= data + capacity) { // loop around if off end
55         front_ptr = data;
56     } // if
57 } // pop()
58
59 template <typename T>
60 T& Queue<T>::front() {
61     return *front_ptr;
62 } // front()
63
64 template <typename T>
65 size_t Queue<T>::size() {
66     if (back_ptr >= front_ptr) {
67         return back_ptr - front_ptr;
68     } // if
69     else {
70         return capacity + back_ptr - front_ptr;
71     } // else
72 } // size()
73
74 template <typename T>
75 bool Queue<T>::empty() {
76     return back_ptr == front_ptr;
77 } // empty()

```

※ 9.3.2 Implementing a Queue Using a Linked List

A queue can also be implemented with a linked list as the underlying container. The process is similar to implementing a stack with a linked list; the only major difference is that elements are added to the *tail* of the linked list rather than the head:



When `.push()` is called, an element is added to the back of the linked list. With a tail pointer, this can be done in $\Theta(1)$ time. When `.pop()` is called, the element at the front of the linked list is removed. The operations for a list-based queue are shown below with their complexities:

Method	Implementation	Complexity
<code>.push(val)</code>	Inserts <code>val</code> to the back of the list	$\Theta(1)$ if tail pointer, $\Theta(n)$ if no tail pointer
<code>.pop()</code>	Delete head node of the list	$\Theta(1)$
<code>.front()</code>	Return reference to data in head node	$\Theta(1)$
<code>.size()</code>	Track size internally as a member variable, or count nodes each time	$\Theta(1)$ if size stored internally, $\Theta(n)$ if nodes counted every time
<code>.empty()</code>	Check if <code>head == nullptr</code>	$\Theta(1)$

The following code implements a queue using a *singly-linked list*:

```
1  template <typename T>
2  class Queue {
3      struct Node {
4          T val;
5          Node* next;
6          Node(T val_in) : val{val_in}, next{nullptr} {}
7      };
8      Node* head;
9      Node* tail;
10     size_t sz;
11 public:
12     Queue();
13     ~Queue();
14     void push(T val);
15     void pop();
16     T& front();
17     size_t size();
18     bool empty();
19 };
20
21 template <typename T>
22 Queue<T>::Queue() : head{nullptr}, tail{nullptr}, sz{0} {}
23
24 template <typename T>
25 Queue<T>::~~Queue() {
26     Node* temp;
27     while (head != nullptr) {
28         temp = head->next;
29         delete head;
30         head = temp;
31     } // while
32 } // ~Queue()
33
34 template <typename T>
35 void Queue<T>::push(T val) {
36     // insert node at tail
37     Node* new_node = new Node{val};
38     if (tail != nullptr) {
39         tail->next = new_node;
40     } // if
41     else {
42         head = new_node;
43     } // else
44     tail = new_node;
45     ++sz;
46 } // push()
```

```

48  template <typename T>
49  void Queue<T>::pop() {
50      // remove node at head
51      Node* temp = head;
52      head = temp->next;
53      --sz;
54      delete temp;
55      if (head == nullptr) {
56          tail = nullptr;
57      } // if
58  } // pop()
59
60  template <typename T>
61  T& Queue<T>::front() {
62      return head->val;
63  } // front()
64
65  template <typename T>
66  size_t Queue<T>::size() {
67      return sz;
68  } // size()
69
70  template <typename T>
71  bool Queue<T>::empty() {
72      return head == nullptr;
73  } // empty()

```

9.4 The STL Queue Container

The C++ standard template library provides a pre-implemented queue container for you in the `<queue>` library. To use this queue in your program, you will need to `#include <queue>` at the top of your code file and declare an object of type `std::queue<>`. A `std::queue<>` supports the following operations:

Function	Behavior
<code>.push(val)</code>	Adds <code>val</code> to the back of the queue
<code>.pop()</code>	Removes the element at the front of the queue (undefined behavior if empty)
<code>.front()</code>	Returns a reference to the element at the front of the queue (undefined behavior if empty)
<code>.size()</code>	Returns the number of elements in the queue
<code>.empty()</code>	Checks if the queue is empty

Similar to a `std::stack<>`, a `std::queue<>` separates element retrieval and removal into two different methods. A call to `.front()` returns the element at the front of the queue, but does not remove it. In contrast, a call to `.pop()` removes the element at the front, but does not return it. The following code goes through an example that utilizes a `std::queue<>`:

```

1  std::queue<int32_t> q;           // initializes a queue with variable name 'q'
2  q.push(5);                     // pushes 5 onto the queue
3  q.push(3);                     // pushes 3 onto the queue
4  int x = q.front();             // x now stores the value 5
5  q.pop();                       // 5 is removed from the queue
6  q.front() = 4;                 // the front element is changed from 3 to 4
7  q.pop();                       // 4 is now removed from the queue
8  std::cout << q.size() << '\n'; // queue is empty, so this prints out 0

```

9.5 Solving Problems Using Stacks and Queues

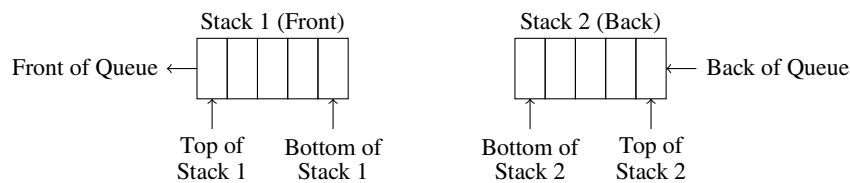
Stacks and queues can be quite useful in solving several different types of programming problems. In this section, we will explore some problems that utilize these containers.

※ 9.5.1 Implementing a Queue Using Two Stacks

Example 9.1 You are given two stacks, each of which supports `.top()`, `.push()`, `.pop()`, and `.size()`. If you are given no other containers, how can you use the two stacks to implement a queue that supports `.front()`, `.push()`, `.pop()`, and `.size()`?

The thing to notice about a stack is that it is a *one-ended* container. Since insertion and removal are both done on the same end, there is no way to access elements at the bottom of a stack. On the other hand, a queue must manage data on both ends. This is why a single stack cannot be used to simulate a queue, since there is only one place to insert and remove data.

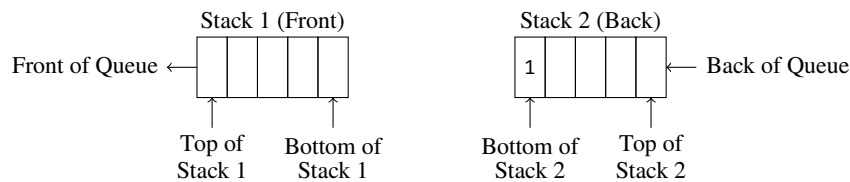
However, this changes if you are given two stacks instead of one. With two stacks, there are now two places where data can be added or removed: the top of stack 1, and the top of stack 2. As a result, we can treat one stack as the "front" of our queue (where elements are removed), and the other stack as the "back" of our queue (where elements are inserted). An illustration of this idea is shown below:



A summary of these operations is shown below, where `stackFront` is the front stack and `stackBack` is the back stack:

```
.push(x): stackBack.push(x);
.front(): return stackFront.top();
.pop(): stackFront.pop();
.size(): return stackFront.size() + stackBack.size();
```

However, this implementation has a slight problem. Suppose we push 1 into our queue and immediately try to pop it. Using the operations above, we would push 1 into our back stack and then pop an element out of our front stack.



However, there is nothing to pop since our front stack is empty! In fact, our queue's `.pop()` and `.front()` methods do not work if all the elements are in the back stack.

To solve this, we will need to move *all* the elements from the back stack to the front stack if `.pop()` or `.front()` is ever invoked while the front stack is empty. We need to remove every element from the back stack because the next element we want to retrieve from our queue is at the bottom of this back stack, and it cannot be accessed before all the elements on top of it are removed first.

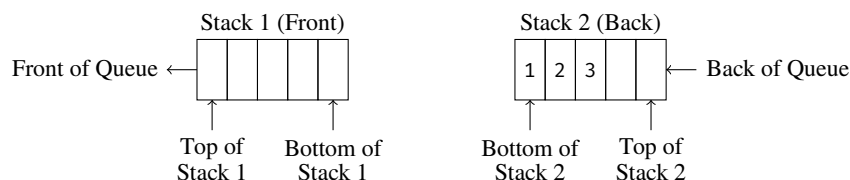
```
QueueWithTwoStacks::front() {
    if (stackFront.empty()) {
        while (!stackBack.empty()) {
            stackFront.push(stackBack.top());
            stackBack.pop();
        } // while
    } // if
    return stackFront.top();
} // front()

QueueWithTwoStacks::pop() {
    if (stackFront.empty()) {
        while (!stackBack.empty()) {
            stackFront.push(stackBack.top());
            stackBack.pop();
        } // while
    } // if
    stackFront.pop();
} // pop()
```

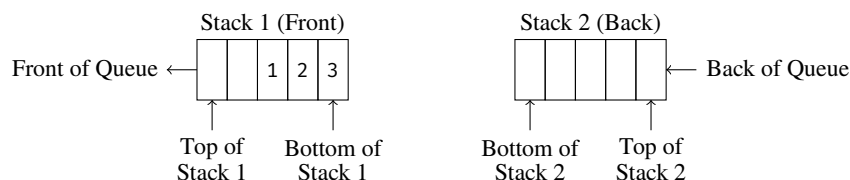
Let's look at how this process works with the following example:

```
1 QueueWithTwoStacks<int> q;
2 q.push(1);
3 q.push(2);
4 q.push(3);
5 q.pop();
6 q.push(4);
7 q.push(5);
8 q.pop();
9 q.pop();
10 q.pop();
```

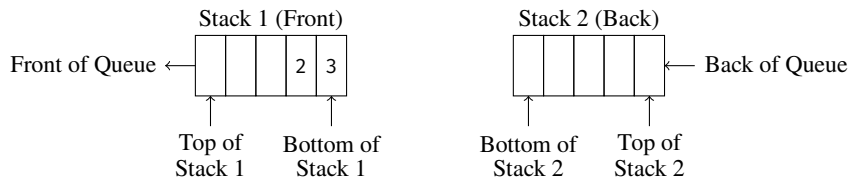
First, we push 1, 2, and 3 to the back of our queue. This is done by pushing these three elements into our back stack:



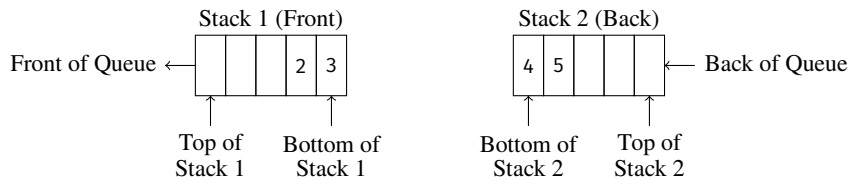
Next, we pop out an element (which should be 1, since it has been in our queue the longest). Since the front stack is empty, we first transfer all the elements from the back stack into the front stack:



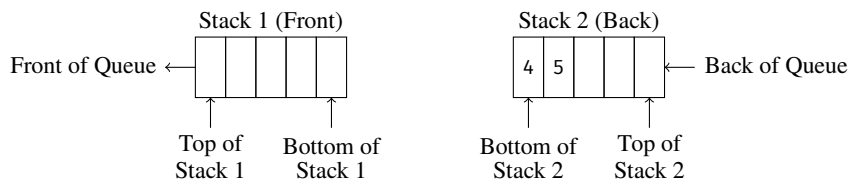
We then pop out the next element in the queue by popping from the front stack:



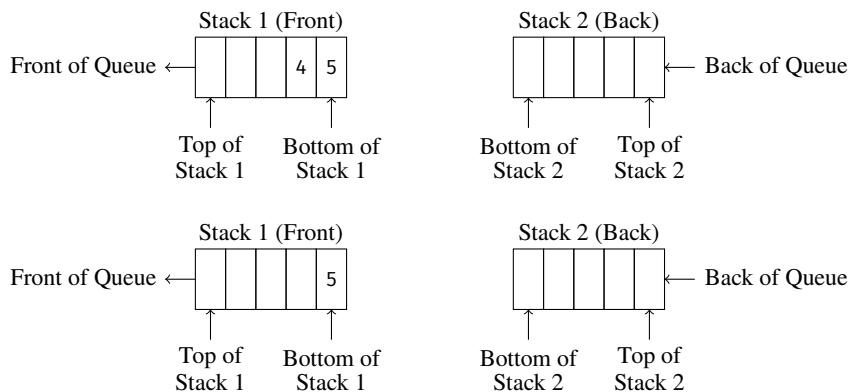
We then push 4 and 5 into our queue, which is added to the back stack:



We then pop out three elements from the queue. Since the front stack is not empty, we can just pop out 2 and 3 without moving anything:



However, for our third pop, the front queue is empty. Thus, we will have to transfer all the elements from the back stack to the front stack before popping out the next element (which is the value 4 in this case):



What's the worst-case time complexity of this queue? Here, `.push()` and `.size()` both take $\Theta(1)$ time, but `.front()` and `.pop()` both take worst-case $\Theta(n)$ time, where n is the size of the queue. This is because these functions may be responsible for moving all the elements from the back stack to the front stack. It may seem that this queue is pretty terrible, since `.front()` and `.pop()` should not be taking linear time! However, looks can be deceiving — even though the worst-case time complexity of a single `.front()` or `.pop()` call may be $\Theta(n)$, this can only happen if the front stack is empty. Using something known as amortized analysis, we can prove that this worst-case scenario happens so rarely that both `.front()` and `.pop()` can essentially be treated as $\Theta(1)$ operations. This concept will be covered in a later chapter.

※ 9.5.2 Sorting a Stack

Example 9.2 Suppose you are given a stack of values, and you are told to sort this stack using only the supported stack operations of `.top()`, `.push()`, `.pop()`, and `.size()`. You are given an additional auxiliary stack (which you may use to help you sort the input stack), but you are not allowed to use any additional containers. How can you solve this problem?

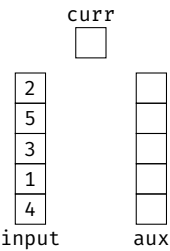
The idea here is to utilize our auxiliary stack to help keep our values sorted. Even though our input stack is not sorted, we can push elements into the auxiliary stack in such a way to keep its elements sorted. This is done by making sure that bigger values are always sent to the bottom of the auxiliary stack. This process is summarized below:

1. Save the value at the top of the input stack in a variable (we will call this value the current element).
2. Since we want larger items to be sent to the bottom of the auxiliary stack, move all elements in the auxiliary stack that are smaller than the current element back into the input stack.
3. Once the auxiliary stack only contains elements larger than the current element, push the current element into the auxiliary stack.

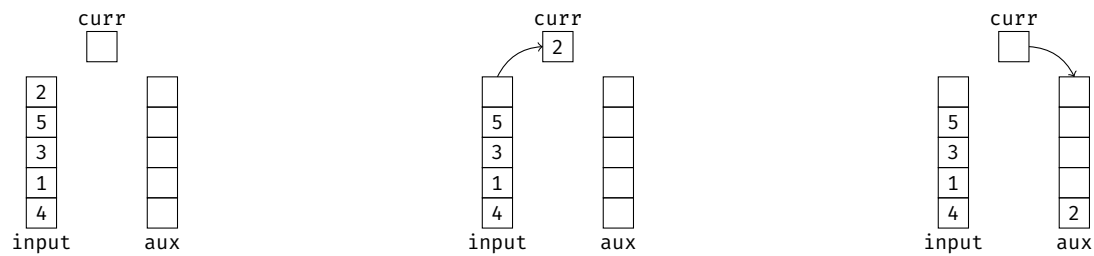
This algorithm is illustrated in the code below:

```
1 while (!input.empty()) {
2   curr = input.top();
3   input.pop();
4   while (!aux.empty() && aux.top() < curr) {
5     input.push(aux.top());
6     aux.pop();
7   } // while
8   aux.push(curr);
9 } // while
```

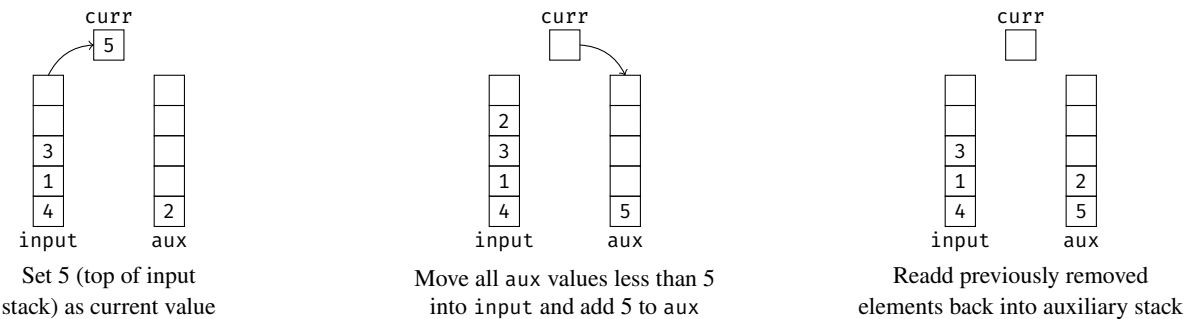
Let’s look at this algorithm in action. In the figure below, input represents the input stack, aux represents the auxiliary stack, and curr represents the current element.



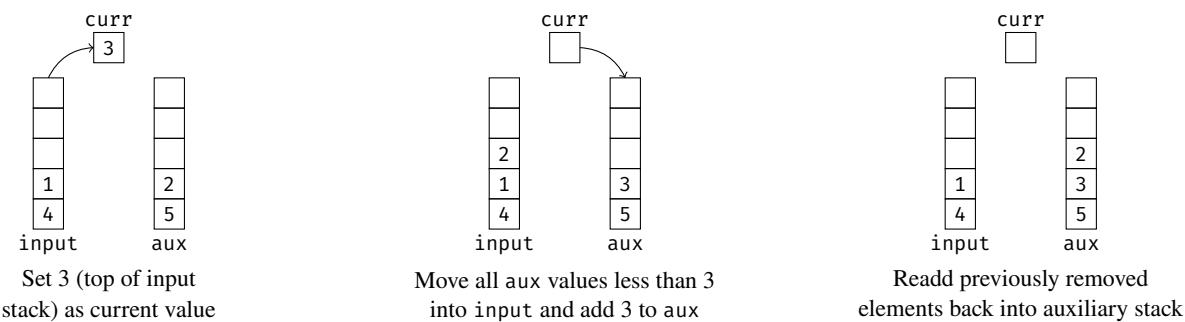
First, we look at the element at the top of the input stack, 2. Since the auxiliary stack is empty, we can immediately push 2 into our auxiliary stack without any problems.



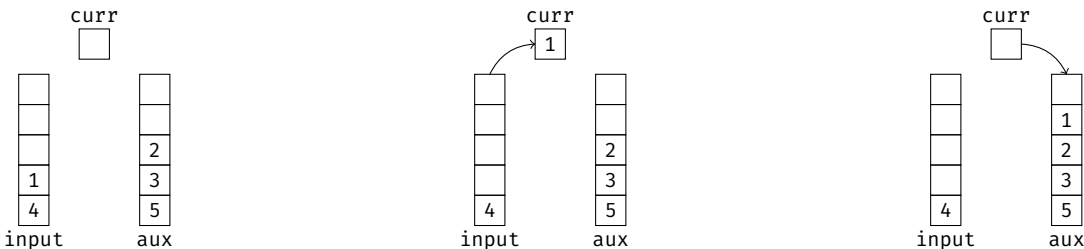
The next element to be considered is 5, which is at the top of the input stack. We want to insert 5 into aux so that it is in the correct sorted position relative to the other values in aux. To do this, we will first move all elements smaller than 5 back into input, push 5 into aux, and reinsert the elements we moved back into aux. In this case, we would move 2 back to input, add 5 into aux, and readd 2 into aux.



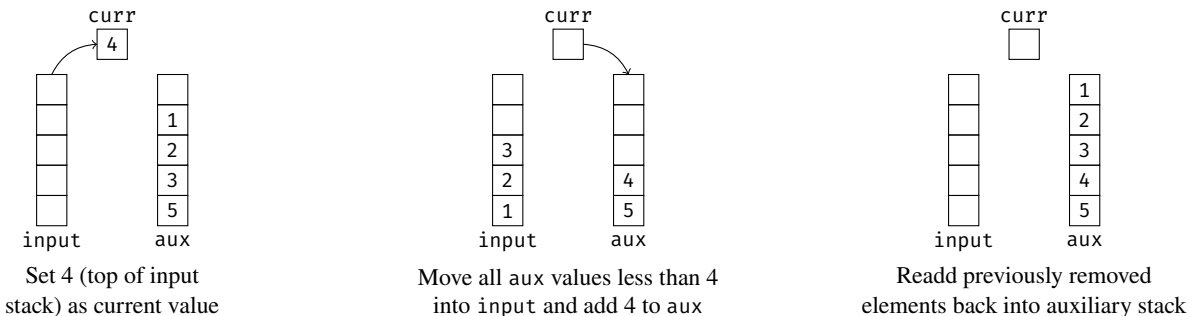
The next element to be considered is 3. To insert 3 into correct sorted position, we first move 2 from the auxiliary stack to the input stack. Then, we insert 3 into the auxiliary stack. Lastly, we add 2 back into the auxiliary stack.



The next element to be considered is 1. Since 1 is already smaller than all the elements in aux, we can just push 1 into aux directly.



The next element to be considered is 4. To insert 4 into correct sorted position, we first move 1, 2, and 3 from the auxiliary stack to the input stack. Then, we insert 4 into the auxiliary stack. Lastly, we add 1, 2, and 3 back into the auxiliary stack.

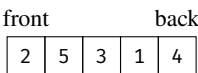


After this algorithm runs to completion, the values in aux will be sorted. The worst-case time complexity for this algorithm is $\Theta(n^2)$, when the input stack is already sorted. When this happens, you will have to move all the elements out of aux whenever you want to push the current element in. For instance, to push the second value into aux, you would need to move out the first element and later readd it in. To push the third value into aux, you would need to move out the first *and* second elements and later readd them in. To push the n^{th} element into aux, you would need to move out all $n - 1$ elements before it and later readd them in. The total number of elements moved in this case would be $1 + 2 + \dots + (n - 1)$, which is $\Theta(n^2)$. The auxiliary space used by this algorithm is $\Theta(n)$, since an additional stack of size n was used to solve this problem.

※ 9.5.3 Sorting a Queue

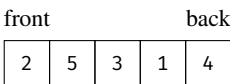
Example 9.3 Suppose you are given a queue of values, and you are told to sort this queue using only the supported queue operations of `.front()`, `.push()`, `.pop()`, and `.size()`. Using no additional containers, how can you solve this problem?

This is similar to the previous problem, but this time you are asked to sort a queue rather than a stack. This difference in container actually makes quite a difference. Unlike a stack, a queue is not single-ended — if you insert elements into a stack, the element at the bottom of the stack cannot be retrieved without first removing and storing the elements above it elsewhere (think about a stack of books: to remove the book at the bottom of the stack, you must first move all the books on top of it). This was why we needed an auxiliary stack for the previous sorting problem. However, to reorder elements in a queue, there is no need to keep track of an auxiliary container. Because data in a queue is inserted and removed on different ends, all elements can eventually be accessed using $\Theta(1)$ auxiliary space by just popping each element off the front of the queue and repushing it onto the back. To demonstrate, suppose we are given the following unsorted queue:

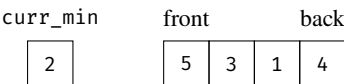


- Assuming that this queue only supports `.push()`, `.pop()`, `.front()`, and `.size()`, we can devise the following algorithm to sort it:
1. Make multiple passes through the queue by popping elements off the front and reinserting them on the back. During each pass, keep track of the smallest value you have seen so far. If you encounter an element that is the smallest you have seen so far, do not push it to the back until a smaller element is found or the pass completes.
 2. At the end of each pass, push the minimum value found to the back of the queue. This value is now in sorted order and should be ignored for all future passes.
 3. Keep on repeating steps 1 and 2 until the queue becomes sorted.

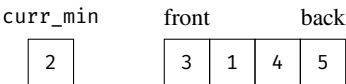
Let's look at this algorithm in action:



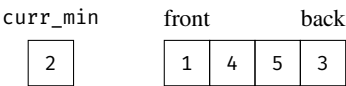
Store the first value in a separate variable that keeps track of the smallest value you have seen on the first pass. After doing so, pop this value off the queue.



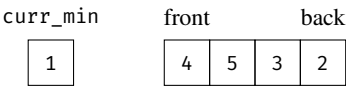
Retrieve the next value from the queue and compare it with the current minimum value encountered. In this case, 5 is not less than 2, so pop it off and push it to the back.



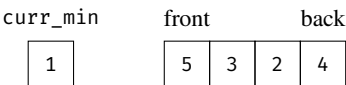
The next value in the queue, 3, is also not less than 2, so it also gets popped off and pushed to the back.



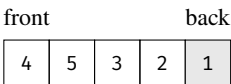
The next value in the queue is 1, which is less than 2. Because we found a new smallest element, we push 2 to the back of the queue and replace curr_min with 1.



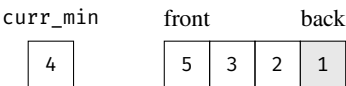
The next value in the queue is 4, which is not less than 1. It gets popped off and pushed to the back.



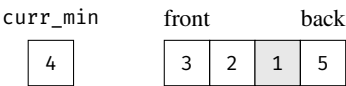
We have now completed our first pass of the queue. Since curr_min is 1 at the end of this first pass, 1 must be the smallest element in the queue. We push 1 to the back of the queue and keep its position fixed for the remainder of the algorithm.



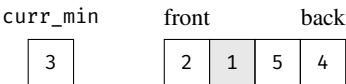
Now, let’s start our second pass. Initialize curr_min to the first value encountered on the second pass, which in this case is 4.



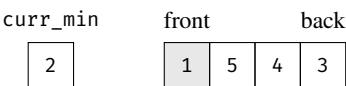
The next value in the queue, 5, is not less than 4. It gets popped off and pushed to the back.



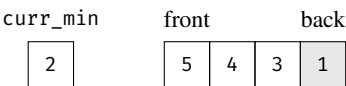
The next value in the queue, 3, is less than 4. Thus, we push 4 to the back and replace curr_min with 3.



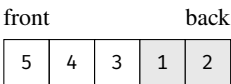
The next value in the queue, 2, is less than 3. Thus, we push 3 to the back and replace curr_min with 2.



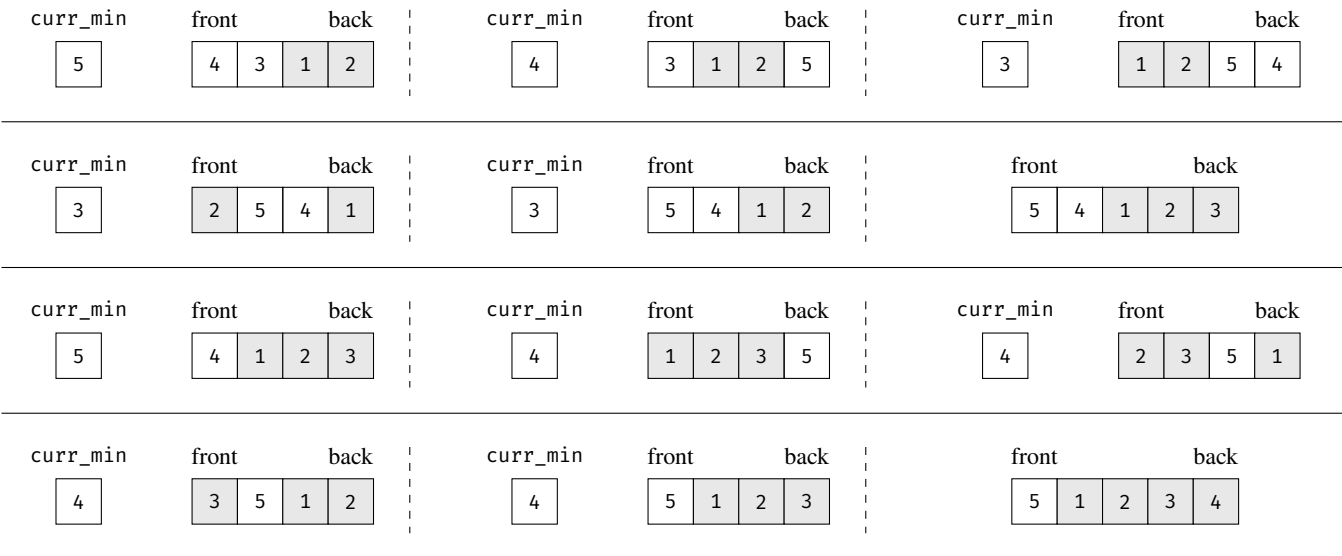
The next value in the queue, 1, has already been considered. Thus, we ignore it, and it gets popped off and pushed to the back.



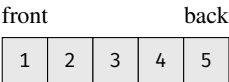
We have finished our second pass of the queue. Since the value of curr_min is 2 at the end of this second pass, 2 must be the second smallest value in the queue. We push it to the back and fix its position.



We can continue this process, which is shown below (read from left to right first, then top to bottom). On the third pass, 3 gets fixed in position. Then, on the fourth pass, 4 gets fixed in position.



At this point, 5 is the last element that hasn't been fixed in position. Because of this, we know that 5 must be the largest element in the queue; we can just pop it off and push it to the end.



Our queue is now sorted. The time complexity of this algorithm is $\Theta(n^2)$. This is because we complete approximately n passes of the queue, where each pass makes n comparisons. As a result, the number of elements we check throughout the lifetime of the algorithm is $\Theta(n \times n)$, or $\Theta(n^2)$. The auxiliary space used by this algorithm is $\Theta(1)$. This is because we did the sorting in-place, using just the input queue itself to complete our algorithm. We did not have to allocate additional memory to solve this problem!

※ 9.5.4 Evaluating Reverse Polish Notation

Example 9.4 Reverse Polish notation is a mathematical notation in which operators follow their operands. For example, if we wanted to add together 1 and 2, reverse Polish notation would express this addition as "1 2 +" instead of "1 + 2". As another example, the expression "(3 - 4) * 5" is written as "3 4 - 5 *" in reverse Polish notation: the "3 4 -" indicates that 3 and 4 should be subtracted, and combining it with "5 *" indicates the this result should be multiplied by 5.

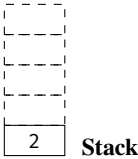
Suppose you are given a valid arithmetic expression in reverse Polish notation (in the form of a vector of strings), with the following valid operators: "+", "-", "*", and "/". Write a function that evaluates the expression's result. You may assume that the expression always evaluates to a solution, and there will never be a situation where you will divide by zero.

Example: Given ["2", "3", "*", "4", "5", "*", "+"], you would return 26, since that is the result of $(2 * 3) + (4 * 5)$.

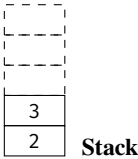
In this problem, each operator acts upon the results of the operands that precede it (for example, the "+" in "2 3 * 4 5 * +" multiplies the result of "2 * 3" with the result of "4 * 5"). Thus, whenever we encounter an operator in our input, we would need to identify the values of the most recent expressions encountered so far (e.g., to apply the "+" operator in our example, we would need to know the values of "2 3 *" and "4 5 *"). A stack would therefore be a good container choice for solving this problem, since it provides this LIFO behavior for identifying these values.

To solve this problem, we will iterate over the input values. Every time we encounter a number, we would push it into a stack. Every time we encounter an operator symbol, we would take out the two values at the top of the stack, apply the operator on these values, and push the result back into the stack. Once we reach the end of the input, the remaining value left in the stack must be the solution of our original expression.

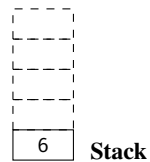
Consider our example input of ["2", "3", "*", "4", "5", "*", "+"]. The first value is the number 2, so we push it onto a stack.



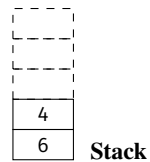
The next value is the number 3, so we push it onto the stack.



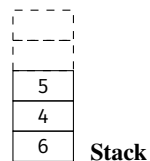
The next value is the operator "*", which indicates that we should multiply the results of the previous two expressions. Thus, we will take out the top two values in our stack (2 and 3), multiply them together, and push in the result of 6 back onto the stack.



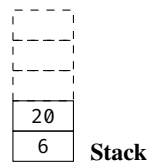
The next value is the number 4, so we push it onto the stack.



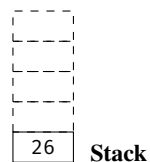
The next value is the number 5, so we push it onto the stack.



The next value is the operator "*", which indicates that we should multiply the results of the previous two expressions. Thus, we will take out the top two values in our stack (4 and 5), multiply them together, and push in the result of 20 back onto the stack.



The next value is the operator "+", which indicates that we should add the results of the previous two expressions. Thus, we will take out the top two values in our stack (6 and 20), add them together, and push in the result of 26 back onto the stack.



There are no more values to process, so the remaining value in the stack, 26, must be our final solution. One solution is shown below:

```

1  // helper function that applies an operator on two values and returns the result
2  int32_t apply_operation(int32_t a, int32_t b, char op) {
3      switch (op) {
4          case '+': return a + b;
5          case '-': return a - b;
6          case '*': return a * b;
7          case '/': return a / b;
8          default: throw std::invalid_argument{"Invalid operation"};
9      } // switch
10 } // apply_operation()
11
12 int32_t evaluate_reverse_polish_notation(std::vector<std::string>& tokens) {
13     std::stack<int32_t> values;
14     for (const std::string& token : tokens) {
15         if ((token == "+" || token == "-" || token == "*" || token == "/")) {
16             int32_t value1 = values.top();
17             values.pop();
18             int32_t value2 = values.top();
19             values.pop();
20             values.push(apply_operation(value2, value1, token[0]));
21         } // if
22         else {
23             values.push(std::stoi(token)); // stoi() converts string to integer
24         } // else
25     } // for token
26     return values.top();
27 } // evaluate_reverse_polish_notation()

```

Since the algorithm traverses over each value in the input vector once, its time complexity is $\Theta(n)$, where n is the size of the input vector. The auxiliary space used by the algorithm is also $\Theta(n)$, due to the additional stack that we allocated to build up our solution (whose size depends on the number of values we have to process).

※ 9.5.5 Balancing Parentheses

One of the classic programming problems that can be solved using stacks is the balanced parentheses problem. In this problem, you are given a string that is made up of the following types of parenthesis characters: `()`, `[]`, and `{}`. Your goal is to write a program that identifies whether the parentheses in the string properly match. Some examples of balanced and unbalanced strings are shown below:

- `"{}[]"` is balanced.
- `"[{()]}"` is balanced.
- `"()([]{})"` is balanced.
- `"{}{}{}{}"` is *not* balanced.
- `"[()]"` is *not* balanced.
- `"(())"` is *not* balanced.

What is the difference between a balanced and unbalanced string? For a string's parentheses to be balanced, each closed parenthesis (i.e., `)`, `]`, or `}`) must match with the most recent open parenthesis (i.e., `(`, `[`, or `{`) that precedes it. Upon first glance, the following solution may seem promising:

1. Keep track of three counters: `left_paren`, `left_brack`, and `left_brace`, one for each type of parenthesis character. Iterate through the string one parenthesis at a time, starting from the left.
2. Whenever an open parenthesis is encountered, increment the counter that keeps track of that parenthesis type.
3. Whenever a closed parenthesis is encountered, attempt to decrement the counter that keeps track of that parenthesis type. If the counter for that parenthesis type is already zero, that means there are no unmatched left parentheses available, and the string must be unbalanced and we exit the loop.
4. If we reach the end of the string and all the counters are exactly zero (i.e., all unmatched left parentheses were matched), then the string must be balanced.

This solution works for some inputs, but it does not work for all. For instance, the following string would lead the previous algorithm to think that it is balanced when it is not.

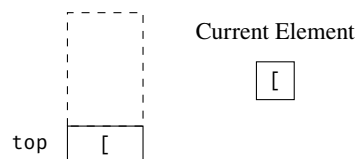
```
str = "[()]"
```

Although the number of times each parenthesis type appears is the same, the *order* of parenthesis types is not valid. Thus, for our algorithm to work for all cases, we will have to ensure that the count *and* position of any open parentheses match the count *and* position of any closed parentheses. To do this, we need a data structure that lends itself well to checking the most recent open parenthesis type seen whenever a closed parenthesis is encountered. It turns out that a stack, with its last-in, first-out data access, is perfect for this type of problem.

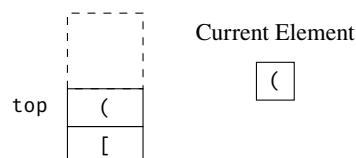
Let's look at how we can solve this problem with a stack. The following algorithm addresses the issue of parenthesis order that broke the previous implementation.

1. Initialize a stack. Iterate through the string one parenthesis at a time, starting from the left.
2. Whenever an open parenthesis is encountered, push it onto the stack.
3. Whenever a closed parenthesis is encountered, check the element at the top of the stack (if the stack is empty after a closed parenthesis is seen, the string is not balanced since there is no matching open parenthesis). If the element at the top is an open parenthesis *of the same type*, pop it off the stack and continue iterating. Otherwise, the order of parentheses is wrong and the string is unbalanced.
4. If the end of the string is reached, check if the stack is empty. If the stack is not empty at the end of the algorithm, there must exist open parentheses that were never matched — in this case, the string must be unbalanced.

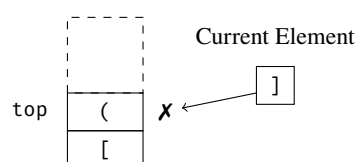
Let's illustrate this using the same string as above (`str = "[()]"`). We first see an open bracket at index 0, so we push it into our stack:



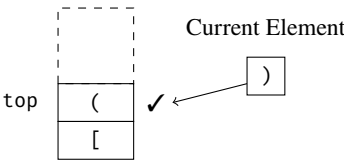
Next, we iterate to index 1 and see an open parenthesis. Since it is open, we push it into our stack:



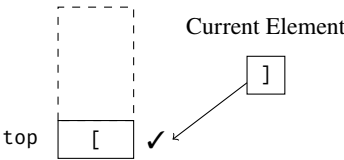
Next, we iterate to index 2 and see a closed bracket. Since it is closed, we compare this bracket with the element at the top of the stack. If the two types were equal (e.g., the top element was also a square bracket), we can continue on with our algorithm. However, in this example, the open parenthesis at the top of the stack is not of the same type as the closed parenthesis we just encountered. Because of this, we immediately know that our string is not balanced. Our algorithm is no longer broken by the order of parentheses!



If our original string were instead `str = [ ( ) ]`, the element at index 2 would be a closed parenthesis rather than a closed bracket. In this case, we have a match, so we would pop the element off the queue and iterate to the next character of the string:



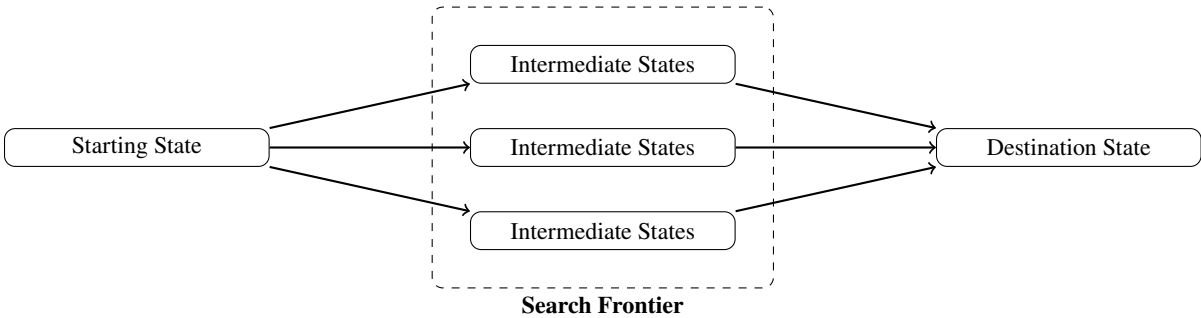
We would then iterate to index 4 and see a closed bracket. Comparing this closed bracket with the element at the top of the stack gives us a match (since both are brackets). We would then pop the bracket off the stack and continue iterating through the string.



Since we finished iterating through the string and the stack is empty, the parentheses in the string must be balanced. Since our algorithm traverses the string once and stores an auxiliary stack of open parentheses, its time and auxiliary space complexity are both worst-case $\Theta(n)$, where n is the length of the string.

9.6 Search Applications of Stacks and Queues

Searching problems play an important role in the field of computer science. These types of problems can appear in many places — from finding the shortest path in a map, to discovering a path out of a maze with obstacles, to solving word ladder style problems where you want to go from one word to another by changing one letter at a time. In a standard searching problem, you are given a starting state, a set of possible next states that are accessible from the current state, and a destination state that you want to reach. To systematically explore a search space to find a viable path from the starting state to the destination state, you can use a stack or queue to hold the set of intermediate states that you have discovered but not yet fully explored (also known as your *search frontier*). The choice of whether to use a stack or a queue to store your search frontier can yield a different order in which the intermediate states are encountered from the starting state to the destination state.



If you use a stack to store your search frontier, you are performing a *depth-first* exploration that rushes down each viable path as far as possible in search of the destination state before backtracking and trying alternatives. Under this searching strategy, all neighboring states that you discover from your current state are pushed into a stack, and the state at the top of the stack (i.e., the one most recently discovered) is always chosen as the state to explore next along your search. This search strategy is formally known as a *depth-first search*, and it can be used to find very deep solutions quickly (i.e., ones that travel through many intermediary states between the starting and destination state), typically with lower memory usage relative to a queue-based solution when paths in the search space are long but narrow.

On the other hand, if you use a queue to store your search frontier, you are performing a *breadth-first* exploration that expansively checks all neighboring states on a level-by-level basis until the destination state is found (i.e., first process all states one intermediary state away from the starting state, then all states two away, then all states three away, etc.). Under this searching strategy, all neighboring states that you discover from your current state are pushed in a queue, and the state at the front of the queue (i.e., the oldest unprocessed state discovered) is always chosen as the state to explore next along your search. This search strategy is formally known as a *breadth-first search*, and it can be used to find the path between the starting and destination states that bypasses the lowest number of intermediary states (also known as a shortest path), provided that the cost of traveling between any two states is constant and unweighted.

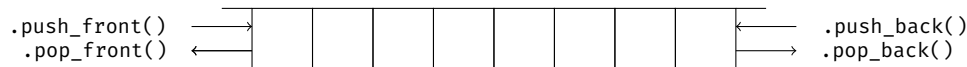
This section is merely a preview of how stacks and queues can be used to solve practical algorithmic searching and pathfinding problems. We will discuss these searching algorithms in greater detail when we cover graph algorithms in chapter 19, so we will not be going into too much additional detail here. However, the following table is provided as a reference that summarizes how these two container types can be used to facilitate different approaches toward exploring a given search space.

Search Container	Stack	Queue
Exploration order	Depth-first (goes deep quickly)	Breadth-first (goes wide and explores outward)
Overall memory usage	Typically lower	Typically higher
Guarantees shortest path?	No	Yes, if unweighted
Typical problem family	Find any path to solution	Find path with minimal number of moves to solution

In general, if you want to explore deeply into each viable path within your search space, you should use a stack to store your search frontier. Alternatively, if you want to explore outward from the starting point on a level-by-level basis, you should use a queue to store your search frontier. In practice, most search and pathfinding problems are built atop either a stack-based (depth-first) or queue-based (breadth-first) exploration strategy with small modifications — this is why an understanding of these two container types is important, as they serve as powerful conceptual keys that can unlock a huge family of searching and problem-solving techniques.

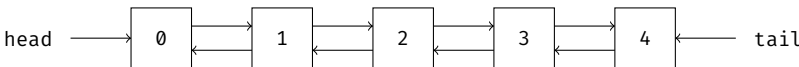
9.7 The STL Deque Container

A **deque** (pronounced as "deck"), or a **double-ended queue**, is an STL container that supports both the functionality of a stack and a queue. Unlike a vector, where efficient insertion and deletion is only supported at the back of the container, a deque supports efficient insertion and deletion operations on *both* ends of the container.



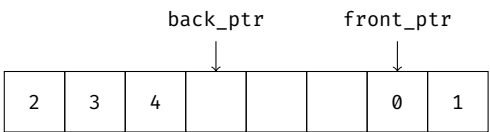
According to the C++ standard, a deque must support $\Theta(1)$ time insertion and removal from both the front (`.push_front()` and `.pop_front()`) and the back (`.push_back()` and `.pop_back()`). Deques must also support $\Theta(1)$ random access using `operator[]`, and insertion and deletion from either end of a deque must *never* invalidate pointers or references to the rest of the elements in the container.

How can a deque be implemented in memory? Since deques must support constant time insertion and deletion from both ends, a linked list implementation may seem tempting at first (as lists support constant time insertion and deletion, assuming that a `tail` pointer is included).



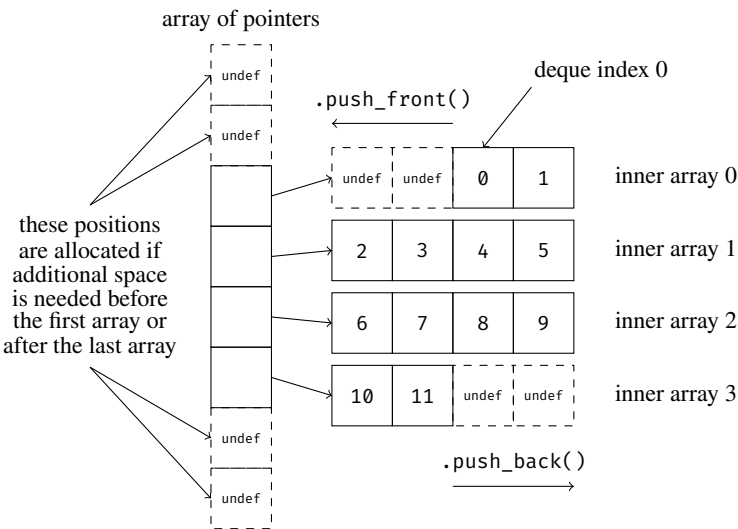
However, this approach does not fully satisfy the C++ standard. This is because a linked list deque cannot support `operator[]` in $\Theta(1)$ time. As mentioned previously, lists do not support random access; if you want to retrieve the n^{th} element of a list, you would have to iterate through all $n - 1$ elements before it. A list is therefore unsuitable for implementing a deque that supports random access.

If linked lists do not work because they cannot provide random access, what about an array? Even though arrays cannot support constant time insertions or deletions from the front, we can remove this problem by using a *circular array* instead. Just like with our circular buffer queue implementation, we will use two pointers to keep track of the front and back of our deque. This allows us to remove and insert elements at the front of our deque without having to shift elements over in our underlying array.



It turns out that the circular array approach does not satisfy the C++ standard either. Even though an array provides random access in constant time, pointer invalidation now becomes a concern. Recall that a deque cannot invalidate any pointers if values are inserted or deleted from the front or back of the container. However, the circular array approach would occasionally require reallocation, which would cause all elements to move to a new location in memory (and thus would invalidate existing pointers).

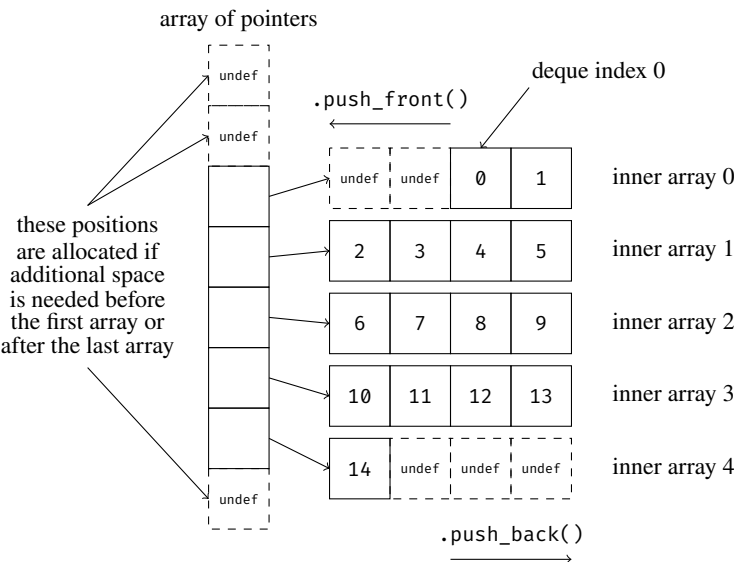
The complex functionality of deques makes them a bit more difficult to implement. We will discuss one common implementation in this section.³ Behind the scenes, a deque is implemented as an array of pointers to other arrays, where each inner array has a constant size (usually a power of two). This is shown in the illustration below:



³The implementation of a deque may be different across libraries, as long as the implementation adheres to the C++ standards for a deque. The implementation covered in this section is taken from a GCC standard library implementation.

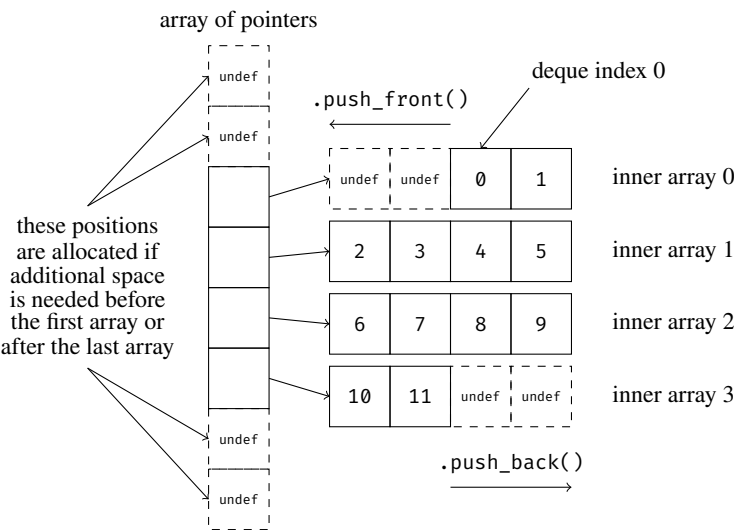
The outer array stores pointers to constant-size inner arrays, and the inner arrays store the deque’s data. The behavior of the inner arrays changes depending on whether the array is at the front or back of the deque. For the last array in the deque, data is added toward the back of the array (e.g., if 12 were pushed to the back of the deque, it would be added to the position directly after 11 in inner array 3). For the first array in the deque, data is added toward the front of the array (e.g., pushing 12 to the front of the deque would add it to the position directly before 0).

If either the inner array at the beginning or end is fully filled, a new inner array is allocated at the next available position of the outer array. For instance, the deque would look like this if 12, 13, and 14 were pushed to the back:



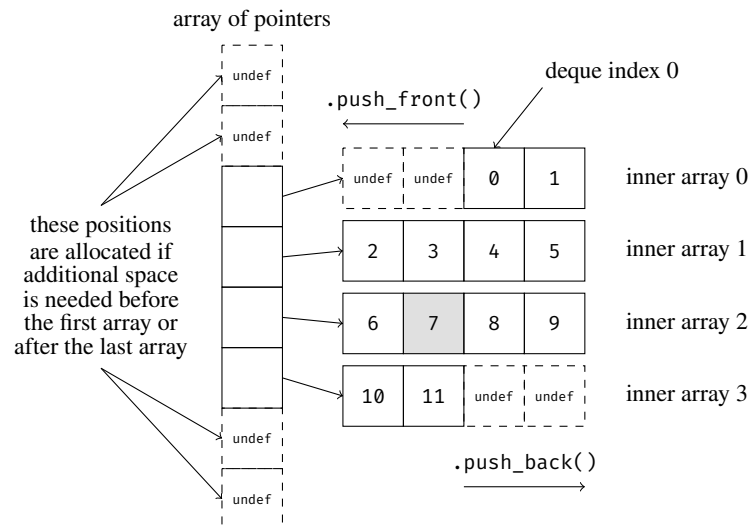
If the entire outer array fills up (i.e., there is no space to allocate new inner arrays at either end), the outer array is reallocated to a larger capacity. This is very similar to how a vector behaves when its underlying array fills up. However, unlike a vector, the additional space from reallocation is split between both ends of the deque’s outer array. This allows insertion to be a constant time operation on both ends of the deque.

This implementation of a deque solves both problems we had when using a circular array and a linked list. Pointer invalidation is no longer an issue when inserting or deleting elements from either end of the container: since the inner arrays do not get reallocated, elements in the deque stay at the same memory location throughout its lifetime (assuming they do not get deleted or moved). This layout in memory also allows the deque to support $\Theta(1)$ random access, which was not possible with a linked list. Given an index, we can quickly find the corresponding element in the deque using some simple arithmetic. For instance, suppose we wanted to retrieve the element at index 7 of the following deque:



- To accomplish this, we first add the index we want (in this case, 7) to the number of unused slots in the first inner array: $7 + 2 = 9$. Then,
- Dividing this result by the capacity of each inner array gives us the array that the target value is in.
 - In this case, we get $9 / 4 = 2$ using integer division (decimals are truncated). This means the element at index 7 is located within inner array 2.
 - Taking the modulo of the original result with the capacity of the inner array gives us the position of the element within its inner array.
 - In this case, we get $9 \% 4 = 1$. This means the element at index 7 is located at index 1 of its inner array.

Thus, the element at index 7 of the provided deque can be found at index 1 of inner array 2 (using zero indexing):



The standard library provides an implementation of a deque for you, defined as a `std::deque<>` in the `<deque>` library. A summary of `std::deque<>` operations is shown below:

```
template <typename T>
std::deque<T>();
```

Default constructor for deque that holds elements of type T, creates an empty deque without any elements.

```
// initializes an empty deque of integers
std::deque<int32_t> d1;
```

```
template <typename T>
std::deque<T>(std::initializer_list<T> init);
```

Initializes the deque with the contents of the initializer list.

```
// initializes d2 and d3 to have contents of {1, 2, 3}
std::deque<int32_t> d2{1, 2, 3};
std::deque<int32_t> d3 = {1, 2, 3};
```

```
template <typename T>
std::deque<T>(const std::deque<T>& other);
```

Copy constructor, copies the contents of other into the constructed deque.

```
// initializes d4 with the contents of d3, or {1, 2, 3}
std::deque<int32_t> d4{d3};
```

```
template <typename T>
std::deque<T>(size_t sz);
```

Creates a deque of sz elements, where each element is value initialized; size equal to sz.

```
// initializes d5 as a deque with size 5, each element initialized to 0
std::deque<int32_t> d5(5);
```

```
template <typename T>
std::deque<T>(size_t sz, T& val);
```

Creates a deque of sz elements, where each element is initialized to a value of val; size equal to sz.

```
// initializes d6 as a deque with size 5, each element initialized to 1
std::deque<int32_t> d6(5, 1);
```

```
template <typename T, typename InputIterator>
std::deque<T>(InputIterator begin_iter, InputIterator end_iter);
```

Creates a deque with all elements in the iterator range `[begin_iter, end_iter)` — inclusive begin but exclusive end. Both `begin_iter` and `end_iter` are input iterators.

```
// initializes d7 with the first two elements of d3, or {1, 2}
std::deque<int32_t> d7(d3.begin(), d3.begin() + 2);
```

Some additional `std::deque<>` operations are summarized below:

template <typename T> size_t <code>std::deque<T>::size();</code>	Returns the number of elements in the deque.
template <typename T> bool <code>std::deque<T>::empty();</code>	Returns whether the deque is empty.
template <typename T> T& <code>std::deque<T>::front();</code>	Returns a reference to the first element (undefined behavior if deque is empty).
template <typename T> T& <code>std::deque<T>::back();</code>	Returns a reference to the last element (undefined behavior if deque is empty).
template <typename T> void <code>std::deque<T>::push_back(const T& val);</code>	Pushes <code>val</code> to the back of the deque, increasing size by 1.
template <typename T, typename... Args> T& <code>std::deque<T>::emplace_back(Args&&... args);</code>	Inserts a new element at the back of the deque, right after the current last element. The new element is constructed in place using arguments for its constructor (<code>args</code>). Returns a reference to the emplaced element since C++17.
template <typename T> void <code>std::deque<T>::pop_back();</code>	Removes the last element in the deque, reducing size by 1 (undefined behavior if deque is empty).
template <typename T> void <code>std::deque<T>::push_front(const T& val);</code>	Pushes <code>val</code> to the front of the deque, increasing size by 1.
template <typename T, typename... Args> T& <code>std::deque<T>::emplace_front(Args&&... args);</code>	Inserts a new element at the front of the deque, right before the current first element. The new element is constructed in place using arguments for its constructor (<code>args</code>). Returns a reference to the emplaced element since C++17.
template <typename T> void <code>std::deque<T>::pop_front();</code>	Removes the first element in the deque, reducing size by 1 (undefined behavior if deque is empty).
template <typename T> iterator <code>std::deque<T>::insert(iterator position, const T& val);</code>	Inserts <code>val</code> directly before the element pointed to by the iterator <code>pos</code> and returns an iterator to the newly added element.
template <typename T> iterator <code>std::deque<T>::insert(iterator position, size_t n, const T& val);</code>	Inserts <code>n</code> copies of <code>val</code> directly before the element pointed to by the iterator <code>pos</code> and returns an iterator to the first new element added.
template <typename T, typename InputIterator> iterator <code>std::deque<T>::insert(iterator position, InputIterator first, InputIterator last);</code>	Inserts a copy of all elements in the iterator range <code>[first, last)</code> directly before the iterator <code>pos</code> and returns an iterator to the first new element added.
template <typename T> iterator <code>std::deque<T>::insert(iterator position, std::initializer_list<T> init);</code>	Inserts the elements in the initializer list into the deque directly before the iterator <code>pos</code> and returns an iterator to the first new element added.
template <typename T, typename... Args> iterator <code>std::deque<T>::emplace(const_iterator pos, Args&&... args);</code>	Inserts a new element directly before the element at position <code>pos</code> . The new element is constructed in place using arguments for its constructor (<code>args</code>). An iterator pointing to the emplaced object is returned.
template <typename T> iterator <code>std::deque<T>::erase(iterator pos);</code>	Erases the element pointed to by the iterator <code>pos</code> and returns an iterator to the element following the one that was erased.
template <typename T> iterator <code>std::deque<T>::erase(iterator first, iterator last);</code>	Erases all elements in the iterator range <code>[first, last)</code> and returns an iterator to the element following the last element that was erased.
template <typename T> void <code>std::deque<T>::clear();</code>	Removes all elements in the container, leaving it with a size of 0.
template <typename T> T& <code>std::deque<T>::operator[](size_t n);</code>	Returns a reference to the element at index <code>n</code> of the deque.

The iterator operations for a deque are similar to those of a vector, where `.begin()` returns an iterator to the beginning of the deque, and `.end()` returns an iterator one past the end of the deque. Deques also support reverse and constant iterators, which can be retrieved using `.rbegin()`, `.rend()`, `.cbegin()`, and `.cend()`.

Function	Behavior
<code>.begin()</code>	Returns a random access iterator to the first element in the deque
<code>.end()</code>	Returns a random access iterator to the position one past the last element in the deque
<code>.cbegin()</code>	Returns a <i>constant</i> random access iterator to the first element in the deque
<code>.cend()</code>	Returns a <i>constant</i> random access iterator to the position one past the last element in the deque
<code>.rbegin()</code>	Returns a <i>reverse</i> iterator to the last element in the deque
<code>.rend()</code>	Returns a <i>reverse</i> iterator to the position one before the first element in the deque
<code>.crbegin()</code>	Returns a constant reverse iterator to the last element in the deque
<code>.crend()</code>	Returns a constant reverse iterator to the position one before the first element in the deque

Deques are similar to vectors in that insertions and deletions from the middle of the container require elements to be shifted after the modification point (to open up a space for insertion, or to bridge a gap for deletion). However, in contrast to a vector, a deque's reallocation may exhibit better overall performance. In a vector, reallocation forces all data in the original array to be copied to a new, larger array. For a deque, reallocation is only done on the outer array of pointers, so only the contents of this outer array are copied during reallocation. This is because the actual data resides in the inner arrays, which have fixed capacity and do not need to be reallocated.

However, even if reallocation may be more efficient for a deque, `operator[]` is more performant for a vector. Despite the fact that `operator[]` is a $\Theta(1)$ operation for both vectors and deques, the constant term for a deque is larger. This is because a vector only needs to perform a simple addition to access a value at any index (i.e., `arr[i] = *(arr + i)`), but deques require a bit more math to get the memory address of the element we want. In addition, elements in a deque are not always contiguous in memory (as shown by the illustrations in this section). As a result, iteration through a deque is typically slower than iteration through a vector — this is because vectors are better at exploiting caching: a phenomenon that allows elements in a sequence to be accessed faster if they are closer together in memory.

9.8 Container Adaptors

Vectors, lists, and deques fall into a category of containers known as *sequence containers*, whose data can be sequentially accessed in an established order. On the other hand, stacks and queues (and priority queues, which will be covered in the next chapter) belong to a class of containers known as **container adaptors**. Container adaptors are not full container classes on their own, but rather interfaces that are adapted for specific needs by modifying or restricting the functionality of a standard container. In the STL, container adaptors do not support iteration.

Previously, we looked at how stacks and queues can be implemented using arrays and linked lists. However, the STL implementations of stacks and queues use neither approach by default. Instead, they are implemented using a deque as the underlying container. When you initialize a `std::stack<>`, you are actually getting a `std::deque<>` that limits data access to only one end. Similarly, if you initialize a `std::queue<>`, you are actually getting a `std::deque<>` that enforces insertion and deletion on different ends. In other words, both the STL stack and queue are built upon an underlying deque that is "adapted" to support only the operations that suit the interface of the desired container.

Thus, if you want to use a stack or a queue in your program, there is functionally no difference between initializing a stack or queue and initializing a deque, as a deque can be used to emulate the functionality of both stacks and queues. If you want a stack, you can initialize a deque and only use `.push_back()` and `.pop_back()`. If you want a queue, you can initialize a deque and only use `.push_back()` and `.pop_front()`. However, it is generally good practice to explicitly declare the container type that matches your needs (i.e., either a stack or a queue, rather than a deque), as it improves the readability of code and reduces the likelihood of mistakes (such as inserting or removing data from the wrong end); a deque should only be chosen if you need double-ended queue behavior that is not supported by a regular stack or queue.

A deque is not the only underlying container that can be used. You can also initialize a stack or a queue using a different underlying container, such as a vector or a list. The underlying container of a stack must support `.push_back()` and `.pop_back()`, while the underlying container of a queue must support `.push_back()` and `.pop_front()`. A table of possible underlying containers is shown below:

	STL Stack (<code>std::stack<></code>)	STL Queue (<code>std::queue<></code>)
Default Underlying Container	<code>std::deque<></code> that only permits <code>.push_back()</code> and <code>.pop_back()</code>	<code>std::deque<></code> that only permits <code>.push_back()</code> and <code>.pop_front()</code>
Optional Underlying Container (must be explicitly specified)	<code>std::list<></code> that only permits <code>.push_back()</code> and <code>.pop_back()</code> — OR — <code>std::vector<></code> that only permits <code>.push_back()</code> and <code>.pop_back()</code>	<code>std::list<></code> that only permits <code>.push_back()</code> and <code>.pop_front()</code>

Notice that a `std::queue<>` cannot be initialized with a `std::vector<>` as the underlying container. This is because vectors do not support `.pop_front()` (and thus cannot provide efficient insertions/removals at different ends of the container). To explicitly specify the underlying container for a stack or a queue, you will need to include the container type directly after the type of the data. For example, the following line of code initializes a stack that uses a `std::vector<>` as its underlying container:

```
std::stack<int32_t, std::vector<int32_t>> s;
```

Similarly, the following code initializes a queue that uses a `std::list<>` as its underlying container:

```
std::queue<int32_t, std::list<int32_t>> q;
```

If you do not explicitly specify the underlying container, a `std::deque<>` will be used by default. For most scenarios, a `std::deque<>` is the ideal underlying container for stacks and queues — hence why the STL uses it as the default!

Practice Quiz

Disclaimer: These practice questions are not official and may not have been vetted for course material. You may use these for practice, but you should prioritize official resources from current staff for a more accurate depiction of what you need to know for assignments and exams.

1. Which of the following statements correctly describes a benefit of using a `std::deque<>` over a `std::vector<>`?
 - A) Deques store their internal data contiguously in memory, while vectors do not
 - B) Using `operator[]` is typically faster on a deque than a vector
 - C) Deques allow values to be efficiently added to both ends of the container, while vectors only support efficient insertion on one end
 - D) The worst-time complexity of finding an arbitrary element is better with a deque than with a vector
 - E) More than one of the above

2. Consider the following snippet of code:


```

1  int main () {
2      std::stack<int32_t> s;
3      s.push(12);
4      s.push(15);
5      s.push(18);
6      s.push(21);
7      s.push(24);
8      s.pop();
9      std::cout << s.top() << '\n';
10 } // main()
```

What does this code output?

 - A) 12
 - B) 15
 - C) 18
 - D) 21
 - E) 24

3. Which of the following is **NOT** a disadvantage of using a linked list over an array to implement a stack?
 - A) The time complexity of pushing in an element is $\Theta(1)$ for an array and $\Theta(n)$ for a linked list
 - B) If you do not internally track the size of a linked list, the time complexity of finding the size of your stack becomes $\Theta(n)$
 - C) Using a linked list is less efficient than using an array, as a linked list must allocate memory for each node individually
 - D) Using a linked list results in higher memory overhead
 - E) None of the above

4. Which of the following real-life situations is most similar to how a queue works?
 - A) Looking for a book on a shelf in the library, where books are sorted alphabetically by title
 - B) Selecting an open seat in the front of the classroom on the first day of class
 - C) Searching for an exam room that is assigned by student ID
 - D) Waiting in line to get seated at a restaurant
 - E) Looking through the midterm exam and answering questions in order of increasing difficulty

5. What is the best possible *time* complexity of removing the most recently added element in a `std::queue<>` of size n and returning the queue with all of the other elements in their original order?
 - A) $\Theta(1)$
 - B) $\Theta(\log(n))$
 - C) $\Theta(n)$
 - D) $\Theta(n \log(n))$
 - E) $\Theta(n^2)$

6. What is the best possible *auxiliary space* complexity of removing the most recently added element in a `std::queue<>` of size n and returning the queue with all of the other elements in their original order?
 - A) $\Theta(1)$
 - B) $\Theta(\log(n))$
 - C) $\Theta(n)$
 - D) $\Theta(n \log(n))$
 - E) $\Theta(n^2)$

7. Suppose you are using a circular buffer with an array to implement a queue. In your implementation, you store two pointers: a front pointer that points to the first element in the queue and a back pointer that points one past the last element in the queue. These pointers wrap around the array as needed. Which of the following indicates that your circular buffer is full, and that you should reallocate your data to a larger array?
 - A) Incrementing the front pointer causes it to point one past the end of the array
 - B) Incrementing the back pointer causes it to point one past the end of the array
 - C) Incrementing the front iterator causes it to point to the same location as the back pointer
 - D) Incrementing the back iterator causes it to point to the same location as the front iterator
 - E) More than one of the above

8. Consider the following snippet of code:

```

1  int main () {
2      MYSTERY_CONTAINER<int32_t> eecs;
3      eecs.push(203);
4      eecs.push(370);
5      eecs.push(281);
6      std::cout << eecs.top() << ", ";
7      eecs.pop();
8      eecs.push(280);
9      eecs.push(376);
10     eecs.push(183);
11     std::cout << eecs.top() << ", ";
12     eecs.pop();
13     std::cout << eecs.top();
14     eecs.pop();
15 } // main()

```

You are told that this code compiles. If the output of this code is "281, 183, 376", what is a valid container type for eecs?

- A) `std::deque<int32_t>`
 - B) `std::stack<int32_t>`
 - C) `std::queue<int32_t>`
 - D) `std::priority_queue<int32_t>`
 - E) None of the above
9. Two retail companies, Darget and Paolmart, keep track of their inventory in different ways. Darget tracks its inventory using the FIFO (first-in, first-out) method, assuming that the goods added to inventory first are also the first removed from inventory for sale. Paolmart tracks its inventory using the LIFO (last-in, first-out) method, assuming that the goods added to inventory last are the first removed from inventory for sale. The two companies want to use an STL data structure to efficiently keep track of their inventories, and they come to you for advice on which data structure to choose. What should you tell these two companies?
- A) Darget should use a queue, and Paolmart should use either a stack or a vector
 - B) Darget should use a stack, and Paolmart should use a queue or a vector
 - C) Paolmart should use a queue, and Darget should use either a stack or a vector
 - D) Paolmart should use a stack, and Darget should use either a queue or a vector
 - E) Both Darget and Paolmart can efficiently use stacks, queues, and vectors to track their inventory
10. Which of the following data structures **CANNOT** be efficiently used as the underlying container for a queue?
- I. `std::vector<>`
 - II. `std::list<>`
 - III. `std::deque<>`
- A) I only
 - B) II only
 - C) III only
 - D) I and II only
 - E) I and III only
11. Which of the following statements is **TRUE** about container adaptors?
- A) Container adaptors rely on the functionality of a standard container (vector, list, deque, etc.) to provide a specific interface
 - B) Stacks and queues are the only two container adaptors that exist in the C++ standard template library
 - C) Container adaptors can only be used on containers that provide constant time random access
 - D) Container adaptors will always support the full functionality of their underlying container
 - E) More than one of the above
12. If you were using a deque in place of a queue in your program, which of the following statements is/are **TRUE**?
- A) If you use `.push_front()` to add data to your queue, you should use `.pop_front()` to remove data from the queue
 - B) If you use `.push_back()` to add data to the queue, you should use `.pop_back()` to remove data from the queue
 - C) If you use `.push_back()` to add data to the queue, you should use `.pop_front()` to remove data from the queue
 - D) `.push_front()` should never be used on your deque if you want to emulate the behavior of a queue
 - E) More than one of the above
13. Which of the following operations **CANNOT** always be done on a deque in $\Theta(1)$ time?
- A) Inserting an element at the front of the deque
 - B) Inserting an element in the middle of the deque
 - C) Inserting an element at the back of the deque
 - D) Removing an element at the front of the deque
 - E) Removing an element at the back of the deque

14. If the integers 1, 2, 3, and 4 are pushed into a queue in this order and then popped out one at a time, in what order will they be removed?
- A) 1, 2, 3, 4
 - B) 1, 2, 4, 3
 - C) 4, 3, 2, 1
 - D) 4, 3, 1, 2
 - E) None of the above
15. Which of the following is a computational application of a queue?
- A) Queues can be used by a program to simulate recursive calls
 - B) Queues can be used by an internet browser to store webpage data for the back and forward buttons
 - C) Queues can be used by a compiler to check for matching braces in a program's code
 - D) Queues can be used by the operating system to allocate scarce resources among different running processes
 - E) None of the above

16. Consider the function:

```
T return_fifth_element(std::stack<T>& s);
```

which returns the fifth oldest element in a stack. For example, if a stack has five elements, the fifth oldest element is the element at the top of the stack. The implementation of this function **only** uses other public methods provided in the `std::stack<>` interface, and it does not access any of the stack's internal data structures directly. When the function returns, the stack *must contain the same elements as before, in the same order*. The fifth oldest element is **NOT** deleted after the function's completion; only its value is returned. If there are fewer than five elements, the newest element in the stack is returned. What is the worst-case time and auxiliary space complexities for the `return_fifth_element()` function, if the most efficient implementation is used? Let n represent the size of the input stack.

- A) Runtime: $\Theta(1)$, Space: $\Theta(1)$
- B) Runtime: $\Theta(1)$, Space: $\Theta(n)$
- C) Runtime: $\Theta(n)$, Space: $\Theta(1)$
- D) Runtime: $\Theta(n)$, Space: $\Theta(n)$
- E) None of the above

17. Consider the following code:

```
1 void mystery_function(std::queue<int32_t>& q) {
2     std::stack<int32_t> s;
3     while (!q.empty()) {
4         int32_t x = q.front();
5         q.pop();
6         s.push(x);
7     } // while
8     while (!s.empty()) {
9         int32_t y = s.top();
10        s.pop();
11        q.push(y);
12    } // while
13 } // mystery_function()
```

What does `mystery_function()` do?

- A) It pops off the element at the back of the input queue
- B) It pops off the element at the front of the input queue
- C) It removes all elements in the input queue
- D) It reverses the input queue
- E) None of the above

18. Consider the following snippet of code:

```
1 struct MysteryContainer {
2     std::queue<int32_t> q;
3
4     void push(int32_t x) {
5         q.push(x);
6         for (size_t i = 0; i < q.size() - 1; ++i) {
7             q.push(q.front());
8             q.pop();
9         } // for i
10    } // push()
11
12    void pop() { q.pop(); }
13    int32_t& top() { return q.front(); }
14    bool empty() { return q.empty(); }
15 };
```

The above code uses a queue to implement the behavior of which data structure?

- A) Queue
- B) Stack
- C) Priority Queue
- D) Deque
- E) None of the above

19. Consider the `MysteryContainer` implemented in the code from the previous question. Suppose you wanted to push n elements into the `MysteryContainer`. What is the time complexity of pushing these n elements into the `MysteryContainer`?
- A) $\Theta(1)$
 - B) $\Theta(n)$
 - C) $\Theta(n^2)$
 - D) $\Theta(n^3)$
 - E) None of the above
20. Which one of the following statements is **TRUE**?
- A) If a `std::deque<>` is used as the underlying container for a `std::stack<>`, then inserting an element into the middle of the stack takes $\Theta(1)$ time
 - B) If a `std::vector<>` is used as the underlying container for a `std::stack<>`, then inserting an element into the middle of the stack takes $\Theta(1)$ time
 - C) If a `std::vector<>` is used as the underlying container for a `std::queue<>`, then inserting an element into the middle of the queue takes $\Theta(1)$ time
 - D) If a `std::list<>` is used as the underlying container for a `std::stack<>`, then inserting an element into the middle of the stack takes $\Theta(1)$ time
 - E) None of the above
21. Which one of the following statements is most accurate about the STL stack container?
- A) The member function `std::stack::push()` will remove data off the top of the stack
 - B) Elements are added to and removed from a stack in first-in, first-out (FIFO) order
 - C) The member function `std::stack::pop()` can be used to directly retrieve data from a stack
 - D) The `std::stack::back()` member function allows a user to access the oldest element in the stack
 - E) Stacks can be implemented using either a vector or a linked list as its underlying container
22. If you want a container that only needs to support the functionality of a `std::stack<>` or `std::queue<>`, which of the following are valid reasons why it may be preferable to select one of these two containers instead of a `std::deque<>`?
- I. Stacks and queues typically perform faster than dequeues, since they are container adaptors that restrict the interface of their underlying container, and thus do not need to support other time consuming operations
 - II. If you only need the behavior of a stack or queue, explicitly using a `std::stack<>` or `std::queue<>` instead of a `std::deque<>` better conveys your intent and may make your code easier to understand and maintain
 - III. Because they are container adaptors that restrict the interface of their underlying container, stacks and queues may be safer to use since they are less prone to programming mistakes (such as popping or pushing from the wrong end)
- A) II only
 - B) I and II
 - C) I and III only
 - D) II and III only
 - E) I, II, and III
23. You want to reverse a singly-linked list. If you are only allowed to use one of these STL data structures in addition to the list you want to reverse, which of the following data structures is **least** useful?
- A) Stack
 - B) Queue
 - C) Dequeue
 - D) Vector
 - E) All of the above data structures are equally useful
24. The STL's queue supports the `std::queue::back()` member function, which returns a reference to the last element in the queue in $\Theta(1)$ time. Knowing this, what is the time complexity of removing the most recently added element in a `std::queue<>` of size n ?
- A) $\Theta(1)$
 - B) $\Theta(\log(n))$
 - C) $\Theta(n)$
 - D) $\Theta(n \log(n))$
 - E) $\Theta(n^2)$
25. Suppose you need a container that optimizes for speed of sequential access over a fixed set of elements. Which of the following STL containers would be the best to use?
- A) `std::vector<>`
 - B) `std::deque<>`
 - C) `std::stack<>`
 - D) `std::queue<>`
 - E) `std::list<>`

26. Which of the following STL data structures can be used to efficiently simulate the behavior of a recursive algorithm without needing a function that makes a recursive call itself?
- I. Vector
 - II. Queue
 - III. Stack
- A) II only
 B) III only
 C) I and III only
 D) II and III only
 E) I, II, and III
27. Which of the following statements is **FALSE** regarding stacks and queues?
- A) The STL stack interface gives direct access to only the most recently inserted element, and not everything else inside the stack
 - B) A vector can be efficiently used as an underlying container for implementing a stack
 - C) A deque can be used to simulate the behavior of both stacks and queues
 - D) If a singly-linked list is used to implement a queue, then the `.size()` member function would always take $\Theta(n)$ time
 - E) None of the above (i.e., all of the above statements are true)
28. A queue is implemented using a non-circular singly-linked list that only supports a head and tail pointer. Let n be the number of nodes in the queue. The `.push()` operation is implemented by inserting a new node at the head of the list, and the `.pop()` operation is implemented by deleting the node at the tail of the list. What is the time complexity of `.push()` and `.pop()` for this queue, if the most efficient implementation is used?
- A) Push: $\Theta(1)$, Pop: $\Theta(1)$
 - B) Push: $\Theta(1)$, Pop: $\Theta(n)$
 - C) Push: $\Theta(n)$, Pop: $\Theta(1)$
 - D) Push: $\Theta(n)$, Pop: $\Theta(n)$
 - E) None of the above
29. Consider the map below:

	1	2	3	4	5	6	7
A	#	#	#	#	#	#	#
B	#			z	#	H	#
C	#		#	#	#		#
D	#			S		w	#
E	#		y		x		#
F	#	#	#	#	#	#	#

- You want to find a path from your starting position (**S**) to your house (**H**). Cells marked with a # are walls that are inaccessible. Assume that you use a **stack**-based pathfinding scheme and add non-visited cells into a `std::stack<>` in the following direction order until you reach **H**: north, east, south, west. Which of the following correctly gives the order in which the cells labeled w , x , y , and z are encountered? For this problem, consider a cell as encountered when it is first pushed onto the pathfinding stack, and assume the search is terminated once **H** is encountered.
- A) x, w, y, z
 - B) x, y, w, z
 - C) y, x, w, z
 - D) y, z, x, w
 - E) Not all of w , x , y , and z are encountered before the destination (**H**) is found
30. Suppose you are implementing a queue using a circular buffer, using two pointers `front_idx` and `back_idx` (with the implementation as defined in section 9.3.1, and replicated below).
- `front_ptr` `back_ptr`

↓ ↓

<code>arr</code>	1	2	3	
------------------	---	---	---	--
- If the underlying array `arr` has a size of 16, which of the following configurations is **NOT** possible?
- A) `front_idx` points to `arr[9]` and `back_idx` points to `arr[9]`
 - B) `front_idx` points to `arr[14]` and `back_idx` points to `arr[11]`
 - C) `front_idx` points to `arr[15]` and `back_idx` points to `arr[16]`
 - D) `front_idx` points to `arr[5]` and `back_idx` points to `arr[8]`
 - E) All of the above configurations are possible

31. Which of the following statements is/are **TRUE**?
- I. By default, a `std::queue<>` uses a `std::deque<>` as its underlying container
 - II. By default, a `std::stack<>` uses a `std::vector<>` as its underlying container
 - III. One difference between a `std::queue<>` that uses a `std::list<>` as its underlying container vs. one that uses a `std::vector<>` is that the list-based queue does not support $\Theta(1)$ time random access, but the vector-based queue does
- A) I only
 B) II only
 C) III only
 D) I and III only
 E) II and III only
32. Given two data structures of the same type and size, we define *interleaving* the two data structures as the process of creating a third data structure of the *same type* with double the size, and where popping elements interleaves the result of popping from each of the two data structures individually. For example, if popping from container A until it's empty yields [4, 8, 10], and popping from container B until it's empty yields [5, 9, 3], then interleaving A and B would produce a container that yields [4, 5, 8, 9, 10, 3] when values are popped. If you are **NOT** allowed to push elements into the two containers you want to interleave (i.e., A and B), which of the following statements is **TRUE**?
- I. Interleaving two `std::vector<>` containers of size n can be done in $\Theta(n)$ time
 - II. Interleaving two `std::stack<>` containers of size n can be done in $\Theta(n)$ time
 - III. Interleaving two `std::queue<>` containers of size n can be done in $\Theta(n)$ time
- A) I only
 B) I and II only
 C) I and III only
 D) II and III only
 E) I, II, and III
33. Implement the following `QueueWithStacks` templated class, which represents a queue using two stacks. The following member functions should be implemented:
- `.size()`: returns the size of the queue time
 - `.empty()`: returns true if the queue is empty and false otherwise
 - `.push()`: inserts an element at the back of the queue
 - `.pop()`: removes an element at the front of the queue

You may use any `std::stack<>` methods, including `.size()`, `.empty()`, `.push()`, and `.pop()`. However, you are **NOT** allowed to use `std::queue<>` in any capacity.

```

1  template <typename T>
2  class QueueWithStacks {
3  private:
4      // These two stacks will be used to simulate the behavior of a queue
5      // You may add other member variables here
6      std::stack<T> stack1;
7      std::stack<T> stack2;
8
9  public:
10     // These methods should be implemented (as efficiently as possible under the constraints)
11     size_t size() const {
12
13     }
14
15     bool empty() const {
16
17     }
18
19     void push(const T& val) {
20
21     }
22
23     void pop() {
24
25     }
26 };

```

34. Implement the following `BrowserTab` class, which simulates a browser tab. You will start on the given homepage (initialized in the constructor), and you can visit different url strings. You are also able to go back or move forward in the URL history a given number of steps. Feel free to add any member variables that you may find useful.

```

1  class BrowserTab {
2  private:
3      // Add any member variables that you may find useful here
4
5  public:
6      // This constructor inits the BrowserTab with the homepage of the browser
7      BrowserTab(const std::string& homepage) {
8
9      }
10
11     // This function visits the given url from the current page
12     // Calling this function clears up all the forward history
13     void visit(const std::string& url) {
14
15     }
16
17     // This function moves back in history a total of "steps" steps (or as far back as possible if the number of
18     // steps specified exceeds the history length) - return the url you end on after moving back in history
19     std::string back(uint32_t steps) {
20
21     }
22
23     // This function moves forward in history a total of "steps" steps (or as far forward as possible if the number
24     // of steps specified exceeds the history length) - return the url you end on after moving forward in history
25     std::string forward(uint32_t steps) {
26
27     }
28 };

```

35. You are given a stack of integers. Implement the following `flip()` method, which flips the order of the elements in the input stack using *only one auxiliary stack* and no other STL or C-style container (although primitive variables are okay).

```
void flip(std::stack<int32_t>& input);
```

You should implement your solution in worst-case $\Theta(n^2)$ time and $\Theta(n)$ auxiliary space, where n is the size of the input stack.

36. Suppose you are given a vector of integers and a number k , and you consider k or more copies of the same value in a row to constitute a "group". You need to sum up any values that are left over after forming and removing groups; the values left over are known as the remnants. For example, given $k = 3$ and the following vector:

```
[3, 5, 5, 2, 2, 2, 5, 5, 5, 2, 3, 4, 4, 4, 3]
```

Starting from the left, there is only one copy of 3 (not a group since there are less than $k = 3$ copies of this value), followed by two 5's (also not a group). These are followed by three 2's, which do form a group, so they are removed from the list:

```
[3, 5, 5, 2, 2, 2, 5, 5, 5, 2, 3, 4, 4, 4, 3]
```

```
[3, 5, 5, 5, 5, 5, 2, 3, 4, 4, 4, 3]
```

Now, there are five copies of the value 5, which do form a group (since the number of 5's is greater than $k = 3$). Thus, these 5's are removed from the list, and the remaining values to be considered are:

```
[3, 5, 5, 5, 5, 5, 2, 3, 4, 4, 4, 3]
```

```
[3, 2, 3, 4, 4, 4, 3]
```

The cluster of 4's is a group, so removing these leaves us with:

```
[3, 2, 3, 4, 4, 4, 3]
```

```
[3, 2, 3, 3]
```

Even though there are k copies of the value 3 remaining, they are not consecutive and thus do not constitute as a group. These four remaining values are the remnants of the original input vector.

Implement the `sum_remnant_values()` function, which takes in a vector of integers `nums` and an integer k , and returns the sum of all remnant values, using the procedure illustrated above. For the given example, the function would return $3 + 2 + 3 + 3 = 11$.

```
int32_t sum_remnant_values(const std::vector<int32_t>& nums, int32_t k);
```

You may assume that `nums` is not empty and that $k > 1$. Note that you should only return the sum of the remnants; you should not physically remove values from the input vector itself. You should implement your solution in worst-case $\Theta(n)$ time and $\Theta(n)$ auxiliary space, where n is the size of the input vector.

37. You are given a string `str` that consists of the characters '(' and ')'. Implement a function that returns the minimum number of parentheses (either '(' or ')') that need to be added to the string so that the resulting string is balanced. For example, given the input string `"()())(())"`, you would return 4, since a minimum of 4 new parentheses are need to balance the string: `"()()()()()"`.

```
int32_t min_add_to_make_string_valid(const std::string& str);
```

You should implement your solution in worst-case $\Theta(n)$ time and $\Theta(n)$ auxiliary space, where n is the size of the input list.

38. Implement the following `MovingAverageCalculator` class, which can be used to calculate the average of the k most recent values that were added. For example, if a calculator is initialized with $k = 3$, and the values of 2 and 4 were added, then `.calculate_k_average()` would return a value of $(2 + 4) / 2 = 3$. If the value 6 were then added afterward, then `.calculate_k_average()` would return a value of $(2 + 4 + 6) / 3 = 4$. Then, if the value 8 were added, `.calculate_k_average()` would return $(4 + 6 + 8) / 3 = 6$ (note that the 2 is no longer considered since it is now the 4th most recently added value, which exceeds our limit of $k = 3$).

```

1  class MovingAverageCalculator {
2  private:
3      // Add any member variables that you may find useful here
4
5  public:
6      // This constructor inits the MovingAverageCalculator with a limit of k
7      MovingAverageCalculator(uint32_t k) {
8
9      }
10
11     // Adds a value to be considered
12     void add_value(double val) {
13
14     }
15
16     // Calculates the average of the k most recent values that were added
17     // If less than k values were added, return the average of all the values
18     double calculate_k_average() const {
19
20     }
21 };

```

» « Quiz Solutions » «

- The correct answer is (C).** Deques allow values to be inserted efficiently at both ends of the container, while vectors only allow efficient insertions at the back. Option (A) is incorrect because vectors do store their data contiguously in memory. Option (B) is incorrect because the contiguity of elements in a vector allows `operator[]` to be implemented with simple pointer addition (deques also require arithmetic, but the more advanced internal structure of a deque results in slightly more complicated arithmetic compared to a vector). Option (D) is incorrect because finding an arbitrary element is worst-case $\Theta(n)$ regardless of which data structure you choose.
- The correct answer is (D).** Five elements are pushed onto the stack. The first `.pop()` on line 8 removes the most recently added element, or 24. Then, the call to `.top()` on line 9 retrieves the most recent element currently on the stack, which is 21.
- The correct answer is (A).** The time complexity of pushing an element into a stack implemented with a linked list is also $\Theta(1)$, since you can simply attach the element to the beginning of the list. All of the remaining options are valid disadvantages of a linked list implementation over an array one.
- The correct answer is (D).** The situation described in answer choice (D) is most similar to how a queue works, since those who are in line first are seated first.
- The correct answer is (C).** To access the most recently added element in a `std::queue<>`, you must remove the $n - 1$ elements that are before it, which takes $\Theta(n)$ time.
- The correct answer is (A).** You do not need any additional space to complete this process: simply pop off the front element and readd it to the back of the queue. Continue doing this for $n - 1$ elements before you reach the final element; at this point, you can take the element out and the other elements will still be in their original order.
- The correct answer is (D).** When you push back an element into the circular buffer, you increment the back iterator; if this back iterator ends up at the same position as the first element in the circular buffer (the front iterator), you would know that your circular buffer is full.
- The correct answer is (B).** When 203, 370, and 281 are pushed into the container, 281 is the first to be retrieved. When 280, 376, and 183 are added to the container, 183 is the first to be retrieved. After 183 is popped off, 376 is the next to be retrieved. Notice that the top value is always the most recent element inserted into the container; thus, of the options provided, the container must be a stack. Notice that a deque would not work here, since they do not support `.push()` and `.pop()` methods without specifying the end you want to push or pop from, so the code would not compile if `MYSTERY_CONTAINER` were a `std::deque`.
- The correct answer is (A).** Since Darget needs to insert inventory from one end of a container and remove it from the other, it should use a queue, which can be used to efficiently support FIFO behavior. On the other hand, Paolmart needs to insert and remove inventory from the same end of a container (since the newest element is removed first), which can be efficiently done using a stack or a vector.
- The correct answer is (A).** Queues need to support efficient insertion and removal from both ends of its underlying container, since the side in which you insert an element is not the side you should pop elements out of. This can be done with a `std::list<>` or a `std::deque<>`, which can support $\Theta(1)$ insertion and removal from both ends of the container, but not with a `std::vector<>`, which only supports $\Theta(1)$ insertion and removal from the back.
- The correct answer is (A).** Only option (A) is a true statement, since it describes the functionality of a container adaptor. Option (B) is false because stacks and queues are not the only container adaptors on the STL (priority queues are another one). Option (C) is false because random access is not a prerequisite for an underlying container: for containers like stacks and queues where random access is not necessary and you only need efficient access at the ends of the container, you can use a `std::list<>` as the underlying container. Option (D) is false, since container adaptors may restrict or modify the functionality of its underlying container to support the desired behavior.

12. **The correct answer is (C).** If you want to emulate the behavior of a queue using a deque, then you should insert and remove elements from opposite ends of the deque.
13. **The correct answer is (B).** Deques support $\Theta(1)$ insertions and removals from the ends of the container, but not the middle. Inserting an element into the middle of the deque may require elements to be shifted in memory, which could take $\Theta(n)$ time.
14. **The correct answer is (A).** The order in which elements are inserted into a queue is the same order in which they are removed.
15. **The correct answer is (D).** Queues are often used to allocate scarce resources among different running processes by providing them in first-in, first-out order for different processes (e.g., if two processes need a shared resource, and one gets there before the other, then the first process would get the resource first). The other three applications are better suited for a stack, which supports efficient insertions and removals from the same end.
16. **The correct answer is (D).** Since you are only allowed to use public methods provided in the `std::stack<>` interface, you would not be able to access the fifth oldest element without popping out all but five of the elements in the stack. Since the order of elements in the stack cannot be changed either, you would need to store the values you popped as well, in an external data structure. Because of this, the worst-case time and auxiliary space complexities of finding the fifth oldest element in a stack are both $\Theta(n)$.
17. **The correct answer is (D).** This function takes all the elements in a queue and inserts them into a stack. Then, it takes the elements in the stack and adds it back to the queue. Since stacks provide LIFO access, the last elements of the original queue are taken out of the stack and reinserted back into the queue first, essentially reversing the order of the queue's contents.
18. **The correct answer is (B).** This class uses a queue to implement the behavior of a stack. This is because, after every insertion, all the previously existing elements in the queue are popped out and reinserted back in the queue. This ensures that the most recently insert element is always at the front of the queue, which essentially recreates the functionality of a stack.
19. **The correct answer is (C).** Each insertion takes $\Theta(n)$ time, since all the elements in the queue are removed and inserted back in. To insert n elements, the total time complexity would become $n \times \Theta(n) = \Theta(n^2)$.
20. **The correct answer is (E).** None of the statements are true. The STL stack and queue do not support constant-time insertions in the middle of the container, regardless of what the underlying container is.
21. **The correct answer is (E).** The underlying container of a stack needs to support $\Theta(1)$ insertion and removal from the same end of the container, which works for both vectors and linked lists. Option (A) is false since pushing an element inserts it into the stack instead of removing it. Option (B) is false because elements are added to and removed from a stack in last in, first out (LIFO) order. Option (C) is false because popping an element out of an STL stack does not return a reference to the removed value. Option (D) is false because no such member exists to access the oldest element in the stack.
22. **The correct answer is (D).** If you only need the functionality of a `std::stack<>` or `std::queue<>`, it may be preferable to use one of these container adaptors rather than a `std::deque<>` since it makes clear in your code what the container is designed to be used for and also lowers the likelihood of programming mistakes (e.g., if you use a `std::queue<>`, you can just use `.push()` and `.pop()` and the container handles it properly for you; for a `std::deque<>`, you would need to keep track of which direction you are inserting and removing elements yourself). Statement I is false because stacks and queues are essentially the same as deques, just with a restricted interface, so there is no difference in performance.
23. **The correct answer is (B).** To build a singly-linked list in reverse order, one common strategy is need to start from the last node and build the list by inserting nodes from the back to the front. This can be done using a container that supports efficient last in, first out (LIFO) access (e.g., push all the nodes into the container, take them out one at a time in LIFO order, and the attach them to the reversed list). Of the four containers provided, all support efficient LIFO access except the queue, which only supports efficient first-in, first-out (FIFO) access.
24. **The correct answer is (C).** The time complexity it takes to *access* the most recently added element is irrelevant here, since you are not changing the contents of the queue. To *remove* the most recently added element from a `std::queue<>`, you have no choice but to first remove and reinsert all the elements before it, which would take $\Theta(n)$ time.
25. **The correct answer is (A).** Sequential access is fastest if the elements to access are stored contiguously in memory. Stacks and queues do not support sequential access at all. Of the remaining three containers, only the `std::vector<>` guarantees that all of its elements are stored contiguously in memory.
26. **The correct answer is (C).** To emulate the stack frames of a recursive call without performing any recursion, you will need a container that supports efficient LIFO access (since with recursion, the item you put on the stack frame most recently is the one you want to retrieve first). This can only be done with a vector and a stack, and not a queue.
27. **The correct answer is (D).** There is nothing that prevents the queue from keeping track of its size internally, even if it uses a singly-linked list as its underlying container. In this case, the time complexity of accessing the queue's size would be $\Theta(1)$, not $\Theta(n)$.
28. **The correct answer is (B).** Inserting an element at the front of a singly-linked list takes $\Theta(1)$ time, and removing an element at the back of a singly-linked list takes $\Theta(n)$ time, regardless of whether a tail pointer is supported. Since push acts upon the front and pop acts upon the back, the time complexity of push would be $\Theta(1)$, and the time complexity of pop would be $\Theta(n)$.

29. **The correct answer is (D).** During the first iteration, D5, E4, and D3 are inserted into the stack, in this order.

	1	2	3	4	5	6	7	
A	#	#	#	#	#	#	#	
B	#			z	#	H	#	
C	#		#	#	#		#	
D	#			S		w	#	D3
E	#		y		x		#	E4
F	#	#	#	#	#	#	#	D5

Stack

D3 is now at the top of the stack, so on the next iteration, we take it out and add the unvisited cells of E3 (y) and D2 into the stack.

	1	2	3	4	5	6	7	
A	#	#	#	#	#	#	#	
B	#			z	#	H	#	
C	#		#	#	#		#	
D	#			S		w	#	D2
E	#		y		x		#	E3
F	#	#	#	#	#	#	#	E4

Stack

D2 is now at the top of the stack, so on the next iteration, we take it out and add the unvisited cells of C2 and E2 into the stack.

	1	2	3	4	5	6	7	
A	#	#	#	#	#	#	#	
B	#			z	#	H	#	
C	#		#	#	#		#	E2
D	#			S		w	#	C2
E	#		y		x		#	E3
F	#	#	#	#	#	#	#	E4

Stack

Continuing this process, we eventually encounter cell z next.

	1	2	3	4	5	6	7	
A	#	#	#	#	#	#	#	
B	#			z	#	H	#	
C	#		#	#	#		#	C2
D	#			S		w	#	E3
E	#		y		x		#	E4
F	#	#	#	#	#	#	#	D5

Stack

	1	2	3	4	5	6	7	
A	#	#	#	#	#	#	#	
B	#			z	#	H	#	
C	#		#	#	#		#	B3
D	#			S		w	#	E3
E	#		y		x		#	E4
F	#	#	#	#	#	#	#	D5

Stack

	1	2	3	4	5	6	7	
A	#	#	#	#	#	#	#	
B	#			z	#	H	#	
C	#		#	#	#		#	B2
D	#			S		w	#	E3
E	#		y		x		#	E4
F	#	#	#	#	#	#	#	D5

Stack

	1	2	3	4	5	6	7	
A	#	#	#	#	#	#	#	
B	#			z	#	H	#	
C	#		#	#	#		#	B4
D	#			S		w	#	E3
E	#		y		x		#	E4
F	#	#	#	#	#	#	#	D5

Stack

The next cell we encounter is x.

	1	2	3	4	5	6	7	
A	#	#	#	#	#	#	#	
B	#			z	#	H	#	
C	#		#	#	#		#	
D	#			S		w	#	
E	#		y		x		#	E5
F	#	#	#	#	#	#	#	D5

Stack

Lastly, while not explicitly shown, we encounter cell w before reaching our destination H.

30. **The correct answer is (C).** Only configuration (C) is impossible, since the back pointer points past the end of the array. In a circular buffer, the back pointer would circle back to the front if it ever moves past the end, so if the last element is at the last index of the array, then back_idx would end up at index 0, and not 16.

31. **The correct answer is (A).** Both `std::stack<>` and `std::queue<>` use a `std::deque<>` as its default underlying container. Statement III is false because a `std::queue<>` does not support random access.

32. **The correct answer is (E).** All three containers can be interleaved in $\Theta(n)$. For the vector, you can just iterate over the two input vectors simultaneously and alternate pushing their values into the interleaved vector. For the queue, you can alternate popping out elements from the two input queues and push them into the interleaved queue. The stack is a trickier since you only have access to the most recently added element, so the items at the top of the two stacks must be pushed into the interleaved stack last. This requires the use of an additional stack; items are popped out of the two input stacks and alternately pushed into this additional stack first, and then these elements are popped back out and pushed into the interleaved stack (which reverses the order of elements to support the stack's LIFO processing order).
33. To solve this problem, we can just use the strategy discussed in example 9.1. We will treat one stack as the front of our queue, and the other stack as the back of our queue. If pop is ever invoked while the front stack is empty, we would move all the elements from the back stack into the front stack. An implementation is shown below:

```

1  template <typename T>
2  class QueueWithStacks {
3  private:
4      std::stack<T> stack1; // front stack
5      std::stack<T> stack2; // back stack
6  public:
7      size_t size() const {
8          return stack1.size() + stack2.size();
9      } // size()
10
11     bool empty() const {
12         return stack1.empty() && stack2.empty();
13     } // empty()
14
15     void push(const T& val) {
16         stack2.push(val);
17     } // push()
18
19     void pop() {
20         if (stack1.empty()) {
21             while (!stack2.empty()) {
22                 stack1.push(stack2.top());
23                 stack2.pop();
24             } // while
25         } // if
26         stack1.pop();
27     } // pop()
28 };

```

34. One way to solve this problem is to use two stacks, one storing back pages and the other storing forward pages. Whenever you visit a page, add it to the back stack and clear out the forward stack. Whenever you want to go back by k pages, pop k pages out of the back stack and into the forward stack, and then set the current page to the one at the top of the back stack. Whenever you want to go forward by k pages, pop k pages out of the forward stack and into the back stack, and then set the current page to the one at the top of the forward stack. One implementation is shown below:

```

1  class BrowserTab {
2  private:
3      std::stack<std::string> pages_back, pages_forward;
4      std::string current_page;
5  public:
6      BrowserTab(const std::string& homepage) {
7          current = homepage;
8      } // BrowserTab()
9
10     void visit(const std::string& url) {
11         pages_forward = std::stack<std::string>();
12         pages_back.push(current);
13         current = url;
14     } // visit()
15
16     std::string back(uint32_t steps) {
17         while (--steps >= 0 && !pages_back.empty()) {
18             pages_forward.push(current);
19             current = pages_back.top();
20             pages_back.pop();
21         } // while
22         return current;
23     } // back()
24
25     std::string forward(uint32_t steps) {
26         while (--steps >= 0 && !pages_forward.empty()) {
27             pages_back.push(current);
28             current = pages_forward.top();
29             pages_forward.pop();
30         } // while
31         return current;
32     } // forward()
33 };

```

35. One challenge of this problem is that you are restricted to only a single auxiliary stack; it is trivial to reverse the order of the elements in another stack (by just popping the elements out of the input stack and pushing it into the other stack), but ensuring that the elements are reversed in the input stack itself is trickier. A general solution to this problem is to repeat the following process from $k = 0$ to $k = n - 1$:
1. Pop off the top of the input stack and store it in a local variable.
 2. Pop $n - k - 1$ elements from the input stack and push it into the auxiliary stack.
 3. Push the element that was popped off in step 1 back onto the input stack.
 4. Pop all elements in the auxiliary stack back into the input stack.

This solution is implemented in code as follows:

```

1  void flip(std::stack<int32_t>& input) {
2      std::stack<int32_t> aux;
3      size_t n = input.size();
4
5      for (size_t k = 0; k < n; ++k) {
6          int32_t next = input.top();
7          input.pop();
8
9          for (size_t j = 0; j < (n - k - 1); ++j) {
10             aux.push(input.top());
11             input.pop();
12         } // for j
13
14         input.push(next);
15
16         while (!aux.empty()) {
17             input.push(aux.top());
18             aux.pop();
19         } // while
20     } // for k
21 } // flip()

```

36. You can use a LIFO container such as a stack or vector to keep track of groups. To do so, iterate over the input values and check the most recently added element to see if it is equal to the new value to be added; if it is, increment a counter associated with that element. Once the counter exceeds k , then you have a group that can be removed — by removing groups while iterating over the values, you are able to handle groups larger than k that are only formed after removing other groups in the input. One implementation is shown below:

```

1  int32_t sum_remnant_values(const std::vector<int32_t>& nums, int32_t k) {
2      struct Counter {
3          int32_t value{};
4          int32_t count{};
5      };
6
7      std::stack<Counter> s;
8      int32_t sum = 0;
9
10     for (int32_t num : nums) {
11         sum += num;
12         if (s.empty()) {
13             s.push({ num, 1 });
14         } // if
15         else if (s.top().value == num) {
16             ++s.top().count;
17         } // else if
18         else {
19             if (s.top().count >= k) {
20                 sum -= s.top().value * s.top().count;
21                 s.pop();
22             } // if
23             if (s.empty() || s.top().value != num) {
24                 s.push({ num, 1 });
25             } // if
26             else {
27                 ++s.top().count;
28             } // else
29         } // else
30     } // for
31
32     // check one more time in case last element caused group to exceed size of k
33     if (s.top().count >= k) {
34         sum -= s.top().value * s.top().count;
35     } // if
36
37     return sum;
38 } // sum_remnant_values()

```

37. One way to solve this problem is to keep counters for the number of left and right parentheses that need to be added for the string to be balanced. We will iterate over the string and count the number of left and right parentheses that are required using the following procedure:
- If a left parenthesis is encountered, increment the number of right parenthesis that are needed (since one will be needed to match this left parenthesis).
 - If a right parenthesis is encountered:
 - If the right parenthesis counter is greater than zero, decrement it.
 - Otherwise, increment the left parenthesis counter, since a new left parenthesis will be needed to match this right parenthesis.

An implementation of this solution is shown below:

```

1  int32_t min_add_to_make_string_valid(const std::string& str) {
2      int32_t left = 0, right = 0;
3      for (char c : str) {
4          if (c == '(') {
5              ++right;
6          } // if
7          else if (right > 0) {
8              --right;
9          } // else if
10         else {
11             ++left;
12         } // else
13     } // for c
14
15     return left + right;
16 } // min_add_to_make_string_valid()

```

This can also be done using a stack. Whenever we encounter a left parenthesis, we push it onto the stack. Whenever we encounter a right parenthesis, we pop it from the stack (or, if the stack is empty, then the right parenthesis has no matching left parenthesis, so we will increment a separate counter). This solution is shown below:

```

1  int32_t min_add_to_make_string_valid(const std::string& str) {
2      int32_t left = 0;
3      std::stack<char> s;
4      for (char c : str) {
5          if (c == '(') {
6              s.push(c);
7          } // if
8          else {
9              if (!s.empty()) {
10                 s.pop();
11             } // if
12             else {
13                 ++left;
14             } // else
15         } // else
16     } // for c
17
18     return left + s.size();
19 } // min_add_to_make_string_valid()

```

38. This problem can be solved using a queue that stores the elements in the window. Each new element is pushed into the back of the queue, and whenever the capacity exceeds k , we pop the oldest element out from the front of the queue. An implementation is shown below:

```

1  class MovingAverageCalculator {
2  private:
3      std::queue<double> window;
4      double sum{};
5      uint32_t window_size{};
6  public:
7      MovingAverageCalculator(uint32_t k) {
8          window_size = k;
9      } // MovingAverageCalculator()
10
11     void add_value(double val) {
12         if (window.size() == window_size) {
13             sum -= window.front();
14             window.pop();
15         } // if
16         sum += val;
17         window.push(val);
18     } // add_value()
19
20     double calculate_k_average() const {
21         return sum / window.size();
22     } // calculate_k_average()
23 };

```